Learning Unitary Transformation from Noisy Quantum States with Deep Complex-Valued Neural Networks on the Stiefel Manifold

Abstract

We present a method for learning unknown unitary transformations from noisy quantum input-output pairs using multi-layer complex-valued neural networks optimized on the complex Stiefel manifold. The multi-layer Cayley update rule ensures monotonic descent of the loss function till convergence, along with keeping all weight matrices unitary throughout training. This overcomes the limitations of existing approaches like intermittent orthogonalization or parameterized quantum circuit methods. We extend this framework to multi-layer networks and demonstrate that increasing depth significantly improves convergence speed, robustness to noise and data efficiency. Experiments on diverse quantum algorithms and quantum circuits with realistic noise show that our method consistently achieves high fidelities under substantial noise levels while preserving exact unitarity at every iteration, establishing state-of-the-art results.

1 INTRODUCTION

Quantum algorithms exploit quantum mechanics to speed up certain computations, such as searching and factoring (Grover [1996],Shor [1994]). Beyond these applications, quantum algorithms are increasingly explored for machine learning (Chen et al. [2020]), leveraging superposition and entanglement in Hilbert spaces. Designing a quantum circuit to realize a desired input–output transformation is fundamentally a unitary operator learning problem, as every physically implementable quantum operation corresponds to a unitary matrix (Nielsen and Chuang [2002]). However, quantum measurements are inherently probabilistic, introducing intrinsic statistical uncertainty. This is further exacerbated by hardware noise, decoherence, and finite sampling, especially on near-term quantum devices (Ge et al. [2024], Shaib et al. [2023]). These challenges have motivated the design of data-driven approaches for quantum circuit synthesis and compilation, including machine learning methods (Houssein et al. [2022]) that learn underlying unitary operators from data. Our work considers the natural problem of learning an unknown unitary transformation from noisy input-output quantum state pairs, enabling quantum circuit design for reconstructing operations from experimental data. Existing approaches such as intermittent orthogonalization (Zomorodi et al. [2024]) and variational quantum process tomography (Xue et al. [2021], Toussi Kiani et al. [2020]), have complementary limitations: the former enforces unitarity only intermittently, causing instability, while the latter, though unitarity-preserving, lacks robustness to noise and its performance depends heavily on the dataset and target operation due to its parameterized structure. Addressing these limitations, we propose a novel framework based on complex-valued neural networks (Hirose [2012]). We optimize the loss function directly on the complex Stiefel manifold of unitary matrices (Absil et al. [2009]) using the Cayley update rule. This guarantees exact unitarity at every training step, enhancing robustness to noisy data, and providing a model architecture that is independent of any specific dataset or target quantum operation. Thus, our approach enables applicability across a broad class of quantum transformations. We summarize our contribution as follows.

Complex-valued multi-layer neural network framework: Our Cayley-based optimization scheme on the complex Stiefel manifold for learning unitary transformations from noisy quantum data is realised within a deep complex-valued neural network. We observe that increased depth enhances convergence and noise resilience. We adapt the Cayley transform to the complex setting, ensuring exact unitarity at every update by treating it as an intrinsic geometric property rather than a periodically enforced constraint. This yields stable and continuous training dynamics and improved robustness to noise, as observed in our experimental results.

Provable convergence, absence of vanishing gradients, and improved data efficiency: A novel feature in our ap-

Submitted to the 42nd Conference on Uncertainty in Artificial Intelligence (UAI 2026). **To be used for reviewing only**.

proach is the composition of multiple unitary layers,

$$U_{\text{total}} = W_{L+1} W_L \cdots W_1, \qquad W_i^\dagger W_i = I.$$

This allows the network to represent rich, structured transformations while preserving exact unitarity. Since the model is linear and each layer is unitary, both forward-propagated states and backpropagated gradients maintain their norm, eliminating vanishing or exploding gradients. We show that for any bounded, Wirtinger differentiable loss and sufficiently small step size, Cayley updates guarantee monotonic descent till convergence while maintaining exact unitarity.

Empirical evaluation and scalability: We compare our Stiefel-manifold optimization with intermittent orthogonalization (Gram–Schmidt, SVD, Lie) and variational process tomography on benchmarks including a Quantum Fourier Transform (QFT) and noisy random circuits of diverse dimensions. Projection-based methods exhibit fidelity–unitarity tradeoffs and oscillatory training, while variational approaches succeed only when the target lies within the chosen ansatz. In contrast, our geometric method maintains exact unitarity and consistently achieves high fidelity. Greater depth improves convergence, robustness, and sample efficiency. Scalability is enhanced by approximating the Cayley inversion with a truncated Neumann series, reducing computational cost while preserving stability.

1.1 RELATED WORK

Research on learning unitary transformations spans quantum computing and machine learning. Two dominant approaches exist: intermittent orthogonalization and parameterized (variational) quantum circuit method. Intermittent orthogonalization periodically projects weights onto the unitary manifold (Zomorodi et al. [2024]), causing non-unitary intermediates and training instabilities. Variational methods parameterize circuits and guarantee unitarity by construction (Xue et al. [2021]), but their expressivity is limited by the chosen ansatz. Alternative parameterization strategies include gradient-based optimization on parameterized gate sets (Toussi Kiani et al. [2020]) and quantum process learning via neural emulation (Marvian and Lloyd [2025]). Recent noisy-learning work jointly trains circuits and noise models, but restricts to pauli noise (Naved et al. [2025]), while channel-learning generalizations operate on density matrices via mixed-unitary models (Belov [2024]).

Unitary neural networks have been extensively studied for recurrent architectures to mitigate vanishing gradients. Early contributions include Unitary Evolution Recurrent Neural Networks (Arjovsky et al. [2016]) and Full-Capacity Unitary RNNs (Wisdom et al. [2016]). More recently, Unitary RNNs employing the scaled Cayley transform were proposed Maduranga et al. [2019]. However these works restricts to real-valued data rather than learning complex-valued quantum unitaries from noisy measurements. Quantum ML approaches to unitary learning rely on variational quantum circuits and remain sensitive to noise (Huang et al. [2021]).

Stiefel-manifold and Cayley-based optimizers have been used for orthogonality constraints (Tagare [2011]), but their systematic application to *complex-valued multi-layer networks* for noisy quantum data is, to our knowledge, novel. Our work fills this gap by combining complex-valued networks, Wirtinger gradients, and Cayley updates with convergence guarantees and empirical quantum noise robustness.

2 PROBLEM FORMULATION

Formally, let n be the number of qubits and assume $N = 2^n$. We are given a set of M noisy training pairs $\{(|\tilde{\psi}\rangle_i, |\tilde{\phi}\rangle_i)\}_{i=1}^M$, where each pair consists of an input state $|\tilde{\psi}\rangle_i \in \mathbb{C}^N$ and an output state $|\tilde{\phi}\rangle_i \in \mathbb{C}^N$. These states are noisy versions of ideal states, such that for an unknown unitary $U \in \mathbb{C}^{N\times N}$, we have $|\tilde{\phi}\rangle_i \approx U|\tilde{\psi}\rangle_i$. The goal is to learn a unitary matrix $W \in \mathbb{C}^{N\times N}$ that approximates U. We design an L-hidden layer complex-valued neural network without biases and with linear activation functions. The l-th ($l \in [1, L+1]$) layer has weight matrix $W_l \in \mathbb{C}^{N\times N}$, and the network's output for an input $|\psi\rangle$ is given by:

$$|\psi_{\text{out}}\rangle = W_{L+1} W_L \cdots W_1 \, |\psi\rangle \, . \tag{1}$$

To ensure that the network represents a unitary transformation, we impose the constraint that each weight matrix is unitary, i.e., $W_l^\dagger W_l = I$ for all l. Since the product of unitary matrices is unitary, the entire network is guaranteed to be unitary. Given the noisy training pairs, we aim to minimize the mean squared error loss function:

$$\mathcal{L}(\{W_l\}) = \frac{1}{M} \sum_{i=1}^{M} \left\| |\tilde{\phi}\rangle_i - W_{L+1} W_L \cdots W_1 |\tilde{\psi}\rangle_i \right\|^2 . \tag{2}$$

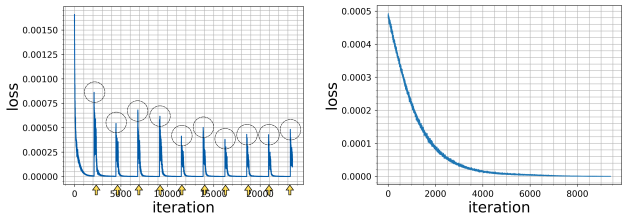
The optimization problem is therefore:

$$\begin{aligned} \min_{\{W_l\}} \quad & \mathcal{L}(\{W_l\}), \\ \text{s.t.} \quad & W_l \in \mathcal{V}_N(\mathbb{C}^N), \quad l = 1, 2, \ldots, L+1. \end{aligned} \tag{3}$$

where $\mathcal{V}_N(\mathbb{C}^N) = \{W \in \mathbb{C}^{N\times N} : W^\dagger W = I\}$ is the complex Stiefel manifold of $N \times N$ unitary matrices.

Learning an unknown unitary transformation from input-output pairs has been currently approached through two complementary paradigms: Intermittent orthogonalization in complex-valued neural networks and Variational methods using parameterized quantum circuits. Understanding their respective strengths and limitations motivates our geometric optimization framework in the Stiefel manifold.

In case of intermittent orthogonalization method, Zomorodi et al. [2024] employ complex-valued neural networks for



(a) Orthogonalization method (b) Ours (on Stiefel manifold)

Figure 1: Training loss vs. iteration comparison between orthogonalization and Stiefel manifold methods. (a) Gram–Schmidt orthogonalization exhibits spikes and unstable training. (b) Cayley update on the Stiefel manifold yields smooth and stable convergence.

quantum circuit design. Their framework learns $U \in \mathbb{C}^{N\times N}$ from clean training pairs $(|\psi_i\rangle, |\phi_i\rangle)_{i=1}^{M}$ where $|\phi_i\rangle = U|\psi_i\rangle$. It implements a single-layer complex-valued network with identity activation, computing $|\psi_{\text{out}}\rangle = W|\psi_{\text{in}}\rangle$ where $W \in \mathbb{C}^{N\times N}$ must satisfy the unitary constraint $W^\dagger W = I$. Training minimizes the mean squared error loss $\mathcal{L}(W) = \frac{1}{M}\sum_{i=1}^{M} || |\phi_i\rangle - W|\psi_i\rangle ||^2$. This approach *recurses* through a two-phase update: perform a sequence of unconstrained gradient descent steps, then orthogonalize W' back to the unitary manifold (e.g., Gram–Schmidt, SVD, or Lie-algebra methods; see Appendix F.2). While this preserves all independent N^2 degrees of freedom prior to orthogonal projection, unitarity is enforced directly on the weight matrix only intermittently. This intermittent projections are seldom meaningful in a loss optimization perspective and hence the periodic projection produces discontinuous trajectories and loss spikes that destabilize training (Fig. 1a). This approach further assumes noiseless data.

In contrast, the variational-circuit paradigm parameterizes the unknown unitary matrix as a quantum circuit realizing $U(\boldsymbol{\theta}) = U_L(\theta_L)\cdots U_1(\theta_1)$ with real parameters $\boldsymbol{\theta}$ optimized to minimize the loss (Xue et al. [2021], Toussi Kiani et al. [2020]). This enforces exact unitarity by construction, but the cascaded parameterized multiplication restricts the independent N^2 degrees of freedom of resultant unitary matrix. Further, the method succeeds only when the target lies in the ansatz's hypothesis space (therefore it is not a universal approximator) and its limited parameter budget makes convergence harder and less robust as system size or data noise increase.

Our method combines the advantages of both paradigms while avoiding their drawbacks. By optimizing directly on the Stiefel manifold using the Cayley update, exact unitarity is preserved at *every* gradient descent, eliminating the discontinuities and loss spikes produced by intermittent projection (Fig. 1b). Unlike fixed circuit ansatz, the method retains the *full* N^2 independent degrees of freedom and learns the weight matrix directly, guaranteeing universal expressivity from any sufficiently informative dataset without assuming a hypothesis class. This combination of exact constraint satisfaction and full expressivity yield stable training, improved robustness to noise, and straightforward extension to leverage the potential of multi-layer architectures.

3 PROPOSED FRAMEWORK

The primary objective of this work is to develop a method for learning unknown unitary transformations from noisy quantum data using complex-valued neural networks. In quantum computing, unitaries represent the fundamental operations that evolve quantum states. However, in practical scenarios, we often have access only to noisy input-output pairs of quantum states, and the exact unitary transformation that relates them is unknown. Our goal is to design a neural network that can learn this unitary transformation from such noisy data, while strictly preserving the unitarity property at every step of the training process. This learned unitary can then be decomposed into a sequence of elementary quantum gates, providing a pathway to implement the transformation on quantum hardware. To achieve this, we propose a complex-valued neural network architecture that *always* resides in the space of unitary matrices (or Stiefel manifold). The network uses identity activation functions and no bias terms, ensuring that each layer performs a linear transformation. By constraining the weight matrices to be unitary, the entire network implements a unitary transformation. The key innovation lies in training this network using optimization on the Stiefel manifold, which guarantees convergence and that the weight matrices remain unitary throughout training, even in the presence of quantum noise.

3.1 OPTIMIZING DEEP COMPLEX VALUED NETWORKS IN THE STIEFEL MANIFOLD

We propose a complex-valued neural network architecture with L hidden layers to learn unknown unitary transformations from noisy quantum data. The network's parameters are unitary matrices, which naturally reside on the complex Stiefel manifold $\mathcal{M}$ (Tagare [2011]):

$$\mathcal{M} = \mathcal{V}_N(\mathbb{C}^N) = \{W \in \mathbb{C}^{N\times N} : W^\dagger W = I\}. \quad (4)$$

The training objective is to minimize a real-valued loss function $\mathcal{L} : \mathbb{C}^{N\times N} \to \mathbb{R}$, which depends on the unitary weights across $L+1$ layers. A fundamental challenge arises from the fact that $\mathcal{L}$ is a non-constant real-valued function of complex variables. Consequently, it does not satisfy the Cauchy–Riemann condition, and hence conventional Euclidean gradient does not exist (Schreier and Scharf [2010]). To address this, we employ Wirtinger calculus, which provides a framework for computing *pseudo* gradients in the complex domain that are suitable for optimization.

Proposition 3.1 (Wirtinger Gradient and its projection)**.** *For a real-valued loss function $\mathcal{L}$ defined on the space of*

complex matrices $W_l \in \mathbb{C}^{N\times N}$(where $1 \le l \le L+1$), the direction of steepest descent is given by the Wirtinger gradient. For layer l at iteration k, it is defined as:

$$G_l^{(k)} = \frac{\partial \mathcal{L}}{\partial \left(W_l^{(k)}\right)^*}, \tag{5}$$

where * denotes the conjugate and $G_l^{(k)} \in \mathbb{C}^{N\times N}$. It provides the steepest descent direction ($-G_l^{(k)}, \forall l$) (see Appendix A), which can be effectively projected onto the Stiefel manifold while following a descent direction.

While the Wirtinger gradient provides a direction in the ambient space, it must be mapped to a unitary-preserving update. A key challenge is that standard gradient descent steps would violate the unitary constraint. To overcome this, we construct a matrix from the Wirtinger gradient and this matrix is then used in a Cayley transform to ensure the updated matrix remains exactly on the Stiefel manifold.

Proposition 3.2 (Complex Cayley Update on the Stiefel Manifold)**.** *Given the Wirtinger gradient $G_l^{(k)}$ and the current unitary weight matrix $W_l^{(k)}$ for layer l at iteration k, a descent direction on the complex Stiefel manifold is generated by the skew-Hermitian matrix:*

$$A_l^{(k)} = G_l^{(k)}(W_l^{(k)})^\dagger - W_l^{(k)}(G_l^{(k)})^\dagger. \tag{6}$$

where $(\cdot)^\dagger$ denotes the complex conjugate transpose (Hermitian conjugate). The unitary weight matrix of layer l of a multilayer network is then updated simultaneously over all layers using a Cayley transform:

$$W_l^{(k+1)} = \left(I + \frac{\lambda}{2}A_l^{(k)}\right)^{-1}\left(I - \frac{\lambda}{2}A_l^{(k)}\right)W_l^{(k)}, \tag{7}$$

where $\lambda > 0$ is the learning rate.

This update guarantees that $W_l^{(k+1)}$ remains exactly unitary at every iteration and that the loss decreases monotonically till it reaches the minima. The Cayley update rule derivation (Eq. (7)) has been given in Appendix B. The multi-layer training procedure is summarized in Algorithm 1.

3.2 INTUITION: HOW MULTI-LAYER HELPS?

Multi-layer unitary networks converge faster and achieve higher fidelity than single-layer networks, especially when learning from noisy data. To understand why, consider a single unitary layer $W \in \mathbb{C}^{N\times N}$. Since W is unitary, it has an orthonormal eigenbasis $\{\mathbf{q}_1, \ldots, \mathbf{q}_N\}$ with eigenvalues $\lambda_i = e^{i\theta_i}$. Any input state Ψ can be written in this basis as:

$$\Psi = \sum_{i=1}^{N} \alpha_i \mathbf{q}_i, \quad \alpha_i = \langle \mathbf{q}_i, \Psi \rangle. \tag{8}$$

Algorithm 1 Multi-layer Stiefel Manifold Optimization

Require: Initial unitary matrices $W_1^{(0)}, W_2^{(0)}, .., W_{L+1}^{(0)} \in \mathcal{V}_N(\mathbb{C}^N)$, learning rate $\lambda > 0$
Ensure: Optimized unitary matrices in Stiefel Manifold $W_1^{(K)}, W_2^{(K)}, .., W_{L+1}^{(K)}$
for $k = 0$ to $K-1$ **do**
  Forward pass: $|\psi'\rangle = W_{L+1}^{(k)}..W_2^{(k)}W_1^{(k)}|\psi\rangle$
  Compute loss: $\mathcal{L} = \frac{1}{M}\sum_{i=1}^{M} \| \, |\psi_{\text{target}}\rangle - |\psi'\rangle \, \|^2$
  Compute gradients: $G_1^{(k)}, G_2^{(k)}, .., G_{L+1}^{(k)}$
  for i, each weight matrix $W_i^{(k)}$ **do**
    Compute skew-Hermitian matrix: $A_i^{(k)} = G_i^{(k)}(W_i^{(k)})^\dagger - W_i^{(k)}(G_i^{(k)})^\dagger$
    Update: $W_i^{(k+1)} = \left(I + \frac{\lambda}{2}A_i^{(k)}\right)^{-1}\left(I - \frac{\lambda}{2}A_i^{(k)}\right)W_i^{(k)}$
  end for
end for
return $W_1^{(K)}, W_2^{(K)}, .., W_{L+1}^{(K)}$

The single unitary layer output is then:

$$\Psi_{\text{out}} = W\Psi = \sum_{i=1}^{N} \lambda_i \alpha_i \mathbf{q}_i \triangleq f(\lambda_W, \mathbf{q}_W; \Psi), \tag{9}$$

which simply rotates the phases of the basis components. Thus a single layer can only move the state along restricted paths determined by its fixed single set of eigenbasis. During training, the network adjusts W to align $W\Psi$ with a target state Φ. Finding the correct alignment with a target state may require many iterations, particularly under noise.

A multi-layer network with L layers applies: $\Psi_{\text{out}} = W_{L+1}W_L \cdots W_1\Psi$, where each W_i is unitary with its own eigenvalues (λ_{W_i}) and eigenbasis ($\mathbf{q}_{W_i}$). The overall transformation Ψ_{out} becomes:

$$f\left(\lambda_{W_{L+1}}, \mathbf{q}_{W_{L+1}}; \ldots f\left(\lambda_{W_2}, \mathbf{q}_{W_2}; f\left(\lambda_{W_1}, \mathbf{q}_{W_1}; \Psi\right)\right)\right). \tag{10}$$

Each layer provides a new eigenbasis, allowing the network to sequentially adjust the state through multiple intermediate representations. This enables exploration of a richer set of directions in state space, effectively allowing coarse adjustments in early layers and finer refinements in later ones. As a result, multi-layer networks navigate more efficiently toward the target, converge in fewer iterations and with lesser data requirement, as confirmed by our experiments.

3.3 CONVERGENCE GUARANTEES

We state a practical convergence result for simultaneous Cayley updates across multiple neural network layers.

Theorem 3.3 (Unitarity, Convergence, and Stability)**.** *Let $\mathcal{L}(W_1, \ldots, W_L)$ be continuously differentiable and*

bounded below function. For sufficiently small step size $\tau > 0$, the simultaneous Cayley updates (Eq. (7)) in all layers satisfy:

1. *each $W_i^{(k)}$ remains unitary for all k;*
2. *$\mathcal{L}(W_1^{(k+1)}, \ldots, W_{L+1}^{(k+1)}) \leq \mathcal{L}(W_1^{(k)}, \ldots, W_{L+1}^{(k)})$ for sufficiently small τ.*
3. *vanishing or exploding gradient problem is avoided, ensuring stable training with convergence even for arbitrarily very deep neural networks.*

The proof of this theorem is given in Appendix C along with the proof of absence of gradient problem in Appendix C.1.

3.3.1 Convergence Visualization via Intrinsic Two-Dimensional Slices of the Stiefel Manifold

For a n-qubit dataset, to gain geometric insight into the optimization dynamics beyond scalar metrics, we propose a visualization scheme in intrinsic two-dimensional slices of the Stiefeld manifold $\mathcal{V}_N(\mathbb{C}^N)$, where $N = 2^n$. Since $\mathcal{V}_N(\mathbb{C}^N)$ is a compact Lie group also, near a target unitary W_{opt}, each iterate can be approximated as:

$$W_k \approx \exp(\Omega_k)\, W_{opt} \tag{11}$$

where Ω_k is a linear combination of standard skew-Hermitian bases (total $\frac{N(N+1)}{2}$). While not every unitary has a skew-Hermitian logarithm, this *approximation* suffices for *visualization*. Expanding Ω_k in a two-dimensional subspace spanned by permuting skew-Hermitian bases A, B gives $\Omega_k = \alpha A + \beta B$ with real α, β. Hence W_k is expressed as:

$$W_k(\alpha, \beta) \approx \exp(\alpha A + \beta B)\, W_{opt} \tag{12}$$

This construction preserves unitarity exactly (as the exponential of a skew-symmetric matrix is unitary) and the corresponding loss surface at 2D coordinates $\{\alpha, \beta\}$ is:

$$\mathcal{L}_{A,B}(\alpha, \beta) = \frac{1}{M} \sum_{i=1}^{M} \|W_k(\alpha, \beta) x_i - y_i\|_2^2 \tag{13}$$

where loss is evaluated on the full dataset to generate the heatmap of the manifold slice, while holding all remaining bases directions fixed. The plotted path (α, β) shows the projection of the true optimization trajectory onto a two-dimensional slice. A detailed mathematical derivation of calculating α, β has been given in the Appendix C.2. In the ideal case without noise, these paths move smoothly toward the origin $(\alpha, \beta) = (0, 0)$, which corresponds exactly to the optimal unitary W_{opt}. This confirms convergence to the global minimum. If the noisy data create a pseudo minima, due to the irreversible loss of information caused by noise, the path would end at a point $(\alpha, \beta) \neq (0, 0)$. To visualize the pseudo minima caused by the noise and to verify convergence still occurs in this case, we propose a second visualization centered at the learned solution at the final iteration (W_{final}). In this case, $W_k \approx \exp(\Omega_k) W_{\text{final}}$, where now the origin represents W_{final}. This reveals the local shape of the loss landscape around the obtained solution and confirms stability of our approach. The key difference is that slices centered at W_{opt} show accuracy of the optimization, while slices centered at W_{final} show stability of the optimization. This distinction matters especially for noisy data, where we need to verify that optimization converges indeed to a stable minima that is created by the noise.

4 EXPERIMENTS AND DISCUSSIONS

We structure our experimental evaluation into two main parts. First, in Section 4.1, we provide an in-depth analysis of our proposed geometric optimization algorithm on the Stiefel manifold, examining its convergence and data requirements, sensitivity to hyperparameters, and scalability. Second, in Section 4.2, we present a comprehensive comparison against a wide range of baseline methods.

For our experiments, we utilize diverse datasets: 2-qubit Bell matrix to capture maximally entangled states, 3-qubit custom data restricting to VQPT ansatz Xue et al. [2021], 6-qubit QFT algorithm, a 5-,7- and a 10-qubit random circuit using all gates, with randomness instilled by seed values to capture nonadherence to any fixed ansatz's hypothesis space. We consider three standard quantum noise models: phase damping, amplitude damping, and depolarizing errors with diverse practical parameters. Noiseless datasets are also included. The generation of training data and the corresponding noise models are described in Appendix F.1.

To evaluate the performance of our method and all baselines, we employ two key metrics. The primary metric is the *fidelity*, which quantifies how well the learned unitary U_{learned} approximates the true target unitary U_{true}. It is defined as:

$$F(U_{\text{true}}, U_{\text{learned}}) \;=\; \frac{1}{N^2} \Big| \operatorname{Tr}\left(U_{\text{true}}^{\dagger} U_{\text{learned}}\right) \Big|^2, \tag{14}$$

where $N = 2^n$ is the Hilbert-space dimension. The fidelity satisfies $0 \leq F \leq 1$, with $F = 1$ indicating perfect recovery up to a global phase. All reported fidelities are averaged over multiple trials. The secondary metric is the *unitarity error*, which measures the numerical deviation of the learned matrix from the unitary manifold:

$$\text{Error}_{\text{unitary}}(W) = \|WW^{\dagger} - I\|_2^2. \tag{15}$$

A properly constrained unitary update should keep this error at or near zero throughout training.

4.1 ANALYSIS OF OUR METHOD

In this section, we dissect the behavior of our Cayley-based optimization on the Stiefel manifold. We present a con-

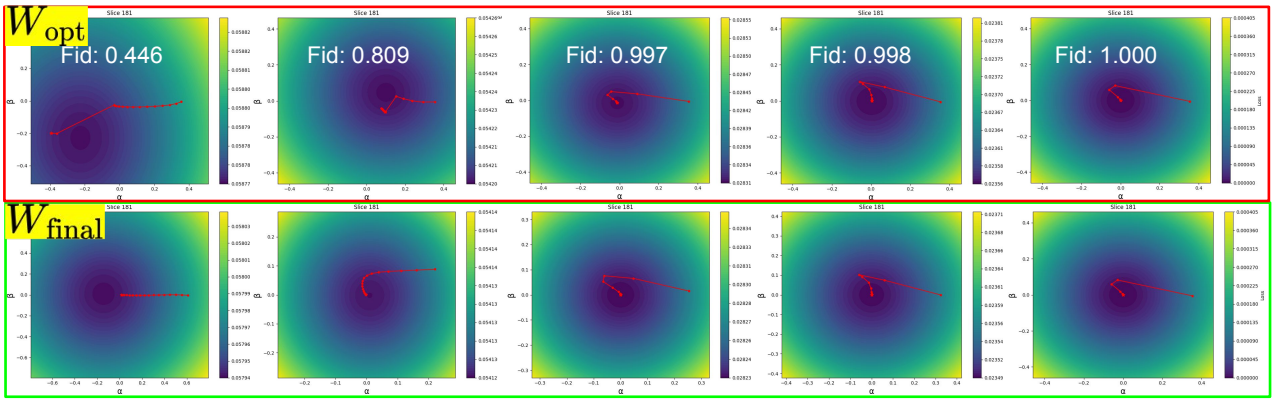


Figure 2: Two-dimensional slices of the Stiefel manifold showing optimization trajectories under different noise levels (high (left) to low(right)). Top row: slices centered at target unitary W_{opt}. Bottom row: same trajectories re-centered at final learned unitary W_{final}. Under high noise, the W_{opt} slice shows convergence to $(\alpha, \beta) \neq (0, 0)$, indicating a different minimizer, while the W_{final} slice confirms smooth local convergence. As noise decreases, both slices converge to the origin.

vergence study visualized on the manifold, followed by an analysis of the effects of network depth, dataset size, learning rate, and initialization. Finally, we demonstrate the scalability of our approach to larger quantum systems.

4.1.1 Convergence Study in the Stiefel Manifold

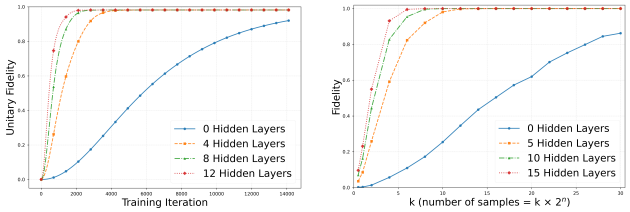
To gain geometric insight into the optimization dynamics, we project the trajectory onto two-dimensional slices of the Stiefel manifold, both centered at the target unitary W_{opt} and final learned unitary W_{final} (Sec. 3.3.1). Figure 2 shows these projections for 5 noise levels (from high to low) on a 5-qubit random circuit under depolarizing noise.

In the slices centered at W_{opt} (top row), we observe that under high noise the trajectory terminates at $(\alpha, \beta) \neq (0, 0)$, indicating convergence to a different minima due to noise-induced distortion of the loss landscape. As noise decreases, the endpoint moves closer to the origin, corresponding to good fidelity. Crucially, when the same trajectories are re-centered at the final learned unitary W_{final} (bottom row), all cases exhibit smooth convergence toward the origin, regardless of noise level. Note that the loss values of W_{opt} is greater thanW_{final} for high noisy cases. The apparent offset in the W_{opt} slice is due to the noise altering the loss landscape rather than instability in the optimization itself. This confirms that the optimization converges to a stable minimum even under high noise. As noise diminishes, the two slices converge to the same trajectory, reflecting the fact that W_{final} approaches W_{opt}. As noise increases, the number of iterations require to converge increases, indicating slower convergence (as depicted by the number of trajectory points in red dots till convergence). This is because high noise induces gradient fluctuations and a rougher landscape, causing longer exploratory paths, whereas low-noise yield smoother landscapes and more direct optimization trajectories. These results illustrate that Stiefel manifold optimization effectively navigates the loss landscape and converges reliably even under significant noise. Robustness to partially corrupted data and corresponding transpiled circuits are analyzed in Appendix E.5, which depict analogous convergence in the hardware realization of the learned unitary.

4.1.2 Effect of Network Depth on Convergence and Data Requirements

One key issue with increasing depth in neural networks is the difficulty in convergence and more data requirements Allen-Zhu et al. [2019] - surprisingly, it is seldom present in our case (see Sec. 3.2). First we examine the impact of network depth on convergence. Experiments are conducted on a 6-qubit QFT dataset of 2000 samples with phase-damping noise ($\lambda = 0.04$), varying the number of hidden layers while keeping other hyperparameters fixed. Figure 3a shows the resulting learning curves. The results reveal a clear trend: increasing depth leads to faster convergence and more efficiently approximate the target unitary. Crucially, the Stiefel method remains stable across all depths, consistently preserving unitarity while achieving high fidelity, hence, confirming the robustness of our geometric approach.

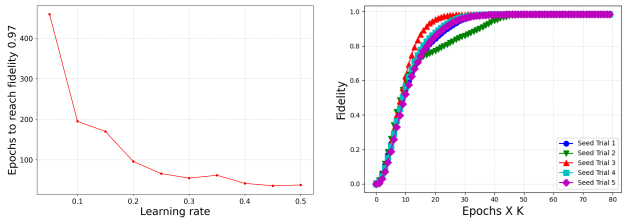
Since quantum measurements are resource-limited, we study how dataset size affects fidelity. Experiments are performed on a 7-qubit random circuit with phase-damping noise ($\lambda = 0.01$). For an n-qubit system, the dataset size is parameterized as $k \times 2^n$, with $k = 0.5, 1, 2,$ etc. We vary network depth keeping all other hyperparameters fixed and record the final fidelity after convergence. As shown in Figure 3b, fidelity increases with dataset size and reaches unity for sufficiently deep networks. Notably, the required k decreases with depth (e.g., $k \approx 12$ for 5 layers, 8 for 10 layers, and 6 for 15 layers), demonstrating that deeper models achieve



(a) Fidelity vs training iterations under noise ($\lambda = 0.04$) for a 6-qubit QFT circuit. Deeper architectures achieve faster convergence and higher final fidelity.

(b) Fidelity vs. k ($k \times 2^n$ samples) under phase-damping noise ($\lambda = 0.01$) for a 7-qubit random circuit with 0, 5, 10, and 15 hidden layers.

Figure 3: (a) Effect of network depth on convergence. (b) Effect of depth on dataset size. Clearly, higher the depth better the convergence and lesser the data requirements



(a) Learning rate sweep

(b) Initialization robustness

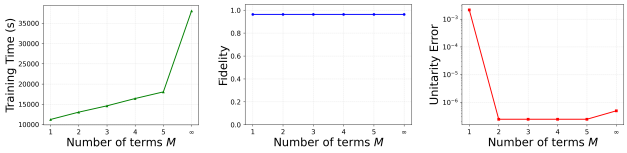
Figure 4: Effect of learning rate and initialization on convergence. (a) Epochs to reach fidelity at least 0.97 vs. learning rate for noisy 6-qubit QFT. Clearly, Stiefel manifold provides a regular loss landscape, and hence admit higher learning rates for faster convergence. (b) Fidelity vs. epoch for noisy 5-qubit random circuit with five random seeds, showing stable convergence regardless of initialization.

the same fidelity with substantially less data requirements.

4.1.3 Effect of Learning Rate and Initialization

Other key challenges of training deep neural networks are the choice of learning rate and weight initialization Cyr et al. [2020]. For a fixed architecture (5 hidden layers) on the 6-qubit QFT task with depolarizing noise ($p = 0.01$, 2000 samples), we vary the learning rate from 0.05 to 0.5. Figure 4a shows that larger learning rates consistently reduce the number of epochs required to reach a target fidelity, without causing instability. This indicates that the Cayley-based updates navigate the loss landscape effectively and consistently find a stable minimum.

To assess robustness to initialization, we train a 5-qubit random circuit under depolarizing noise ($p = 0.08$) using five different random seeds. Figure 4b plots fidelity versus epoch for each seed. Despite starting from distinct random unitary matrices, all trials converge to near-perfect fidelity, demonstrating that the optimization is insensitive to initialization and consistently finds a good minimum. Together, these results confirm the stability and reliability of our geometric optimization framework across different learning rates (higher the better) and random initializations.



(a) Training time

(b) Fidelity

(c) Unitary error

Figure 5: Scalability study showing (a) training time, (b) fidelity, and (c) unitary error as functions of the number of Neumann-series terms M (= 2 or 3 gives a good tradeoff).

4.1.4 Scalability to Large Quantum Systems

Our experimental results demonstrate effectiveness of the proposed Cayley-based optimization on quantum systems on par with the competing methods (e.g., 8 qubits which corresponds to unitary matrices in $\mathbb{C}^{256\times256}$). However, contemporary quantum devices routinely operate on larger scales. The Cayley update requires computing $\left(I + \frac{\lambda}{2}A_l^{(k)}\right)^{-1}$ (see Eqn. (7)), which can become a bottleneck for large N (i.e., $\mathcal{O}(N^3)$) Embree et al. [2025]. (Pl refer to Appendix D for a complexity analysis). However, our exact form of Cayley update can come to the rescue. Since the learning rate λ is *not* very high, the matrix $\frac{\lambda}{2}A_l^{(k)}$ has a small norm, enabling approximation via Neumann series expansion. For any matrix X with $\|X\| < 1$, we have $(I + X)^{-1} = \sum_{m=0}^{\infty}(-X)^m$. Truncating after M terms yields an approximate inverse computable with only matrix multiplications, reducing per-iteration complexity from $\mathcal{O}(N^3)$ to $\mathcal{O}(MN^2)$, where $M \ll N$, that is:

$$W_l^{(k+1)} \approx \left(\sum_{m=0}^{M}\left(-\frac{\lambda}{2}A_l^{(k)}\right)^m\right)\left(I - \frac{\lambda}{2}A_l^{(k)}\right)W_l^{(k)}. \tag{16}$$

As shown in Fig. 5, we evaluate the impact of the Neumann series truncation order M on a 10-qubit random unitary learning task. The training time (Fig. 5a) grows modestly from $M = 1$ to $M = 5$, but jumps dramatically for the exact inverse ($M = \infty$). Crucially, the fidelity (Fig. 5b) remains consistently high (≈ 0.96) across all M, including the exact case. The unitary error (Fig. 5c) is slightly elevated at $M = 1$ (2×10^{-3}), but drops to negligible levels (1×10^{-6}) for $M \geq 2$. Our results demonstrate that low-order truncation ($M = 2$ or 3) often suffices due to rapid decay of higher-order terms. Additional scalability can be achieved by leveraging multi-GPU acceleration Zhang and Gao [2015]. These enhancements ensure our framework remains viable as quantum hardware scales to higher qubits.

Method	No noise		Phase damping				Amplitude damping				Depolarizing			
			$\lambda = 0.01$		$\lambda = 0.04$		$\gamma = 0.02$		$\gamma = 0.05$		$p = 0.03$		$p = 0.06$	
	Fid.	**Err.**	**Fid.**	**Err.**	**Fid.**	**Err.**	**Fid.**	**Err.**	**Fid.**	**Err.**	**Fid.**	**Err.**	**Fid.**	**Err.**
Epoch 300 (Immediately after unitarity constraint)														
GS	$0.811 \pm .13$	10^{-15}	$0.566 \pm .26$	10^{-14}	$0.376 \pm .30$	10^{-14}	$0.355 \pm .29$	10^{-14}	$0.195 \pm .25$	10^{-14}	$0.285 \pm .30$	10^{-14}	$0.142 \pm .20$	10^{-14}
MGS	$0.819 \pm .14$	10^{-15}	$0.572 \pm .26$	10^{-14}	$0.360 \pm .30$	10^{-14}	$0.348 \pm .30$	10^{-14}	$0.192 \pm .25$	10^{-14}	$0.282 \pm .30$	10^{-14}	$0.144 \pm .20$	10^{-14}
SVD	$0.899 \pm .13$	10^{-14}	$0.721 \pm .19$	10^{-14}	$0.491 \pm .28$	10^{-14}	$0.481 \pm .28$	10^{-14}	$0.257 \pm .31$	10^{-14}	$0.379 \pm .30$	10^{-14}	$0.208 \pm .28$	10^{-14}
Lie	$0.104 \pm .02$	10^{-15}	$0.075 \pm .03$	10^{-15}	$0.026 \pm .02$	10^{-15}	$0.043 \pm .02$	10^{-15}	$0.015 \pm .00$	10^{-15}	$0.032 \pm .02$	10^{-15}	$0.012 \pm .00$	10^{-15}
Epoch 301 (Post-constraint drift)														
GS	$0.998 \pm .00$	15.8 ± 14	$0.828 \pm .03$	13.8 ± 11	$0.502 \pm .12$	13.7 ± 10	$0.474 \pm .12$	13.6 ± 9.9	$0.192 \pm .14$	13.8 ± 9.1	$0.341 \pm .18$	13.8 ± 11	$0.142 \pm .13$	13.6 ± 11
MGS	$0.999 \pm .00$	15.3 ± 13	$0.810 \pm .04$	13.4 ± 9.6	$0.501 \pm .12$	14.1 ± 10	$0.501 \pm .12$	14.1 ± 10	$0.197 \pm .14$	14.0 ± 10	$0.347 \pm .17$	13.7 ± 11	$0.139 \pm .13$	13.8 ± 11
SVD	$0.996 \pm .00$	15.6 ± 14	$0.827 \pm .04$	13.1 ± 9.4	$0.493 \pm .14$	14.0 ± 10	$0.488 \pm .13$	13.8 ± 10	$0.222 \pm .17$	13.6 ± 9.7	$0.346 \pm .17$	13.6 ± 10	$0.160 \pm .19$	13.5 ± 10
Lie	$0.878 \pm .24$	22.3 ± 14	$0.733 \pm .18$	15.5 ± 10	$0.371 \pm .12$	14.2 ± 10	$0.351 \pm .11$	14.2 ± 10	$0.115 \pm .04$	13.6 ± 10	$0.219 \pm .08$	13.8 ± 11	$0.076 \pm .03$	13.3 ± 11
Ours	$\mathbf{1.0 \pm .00}$	$\mathbf{2 \times 10^{-11}}$	$\mathbf{0.997 \pm .00}$	$\mathbf{2 \times 10^{-11}}$	$\mathbf{0.970 \pm .02}$	$\mathbf{2 \times 10^{-10}}$	$\mathbf{0.968 \pm .01}$	$\mathbf{2 \times 10^{-10}}$	$\mathbf{0.822 \pm .08}$	$\mathbf{2 \times 10^{-10}}$	$\mathbf{0.951 \pm .01}$	$\mathbf{2 \times 10^{-10}}$	$\mathbf{0.718 \pm .11}$	$\mathbf{2 \times 10^{-10}}$

Table 1: Comparison of fidelity and unitary error at immediate re-orthogonalization) vs. post-constraint. Clearly there is fidelity-unitarity trade-off between constraint enforcement and post-constraint, which our method addresses.

Method	Custom		Random Rx, Ry, Rz, CX		6-qubits QFT		7-qubits Random	
	No Noise	Noisy	No Noise	Noisy	No Noise	Noisy	No Noise	Noisy
VQPT	0.999	0.949	0.086	0.076	0.084	0.070	0.025	0.022
Givens	0.940	0.924	0.046	0.043	0.050	0.05	0.012	0.012
Lie	0.999	0.922	0.116	0.098	0.126	0.124	0.033	0.025
Ours	**0.999**	**0.990**	**0.998**	**0.995**	**1.000**	**0.997**	**0.987**	**0.972**

Table 2: Fidelity comparison of parameterised circuit method. Clearly, these methods seldom work for arbitrary dataset, and are not designed for noisy scenarios.

4.2 BASELINE COMPARISONS

We compare our Stiefel manifold optimization against both families of baseline methods existing in the literature: Intermittent orthogonalization techniques and Parameterized quantum circuit (PQC) approaches.

Intermittent Orthogonalisation Methods: These baselines intermittently enforce unitarity during unconstrained gradient descent using Gram–Schmidt (GS), Modified Gram–Schmidt (MGS), SVD, and a Lie-based projector. Experiments use a 6-qubit QFT dataset (2000 samples per noise level; Section 4) and report average fidelity and unitary error across learning rates $\{0.001, 0.05, 0.1, 0.15, 0.2\}$.

As shown in Table 1, with projections on every 10 epochs the methods exhibit a fidelity–unitarity tradeoff: At epoch 300, while the standard methods achieve a low unitary error (10^{-14}), their fidelity is significantly degraded. Conversely, at epoch 301, although fidelity appears to recover, the unitary error spikes to unacceptably high levels (e.g., ~ 15 for GS!). This oscillation suggests that standard orthogonalization methods are inherently unstable in noisy landscapes. In contrast, our *method* consistently maintains high fidelity across all noise models while keeping the unitary error approximately in order of 10^{-10} irrespective of the number of epochs. Note that the comparably high unitarity of our method can be attributed to cascaded weight multiplications across $L + 1$ layers, however we show that this level of precision is sufficient for successful transpilation.

Parameterized Quantum Circuit Methods: We benchmark our Stiefel-manifold optimization against three PQC baselines: (i) the VQPT ansatz (Xue et al. [2021]) (single-qubit R_z-R_y-R_z blocks with staggered CNOT layers), (ii) a dense Givens decomposition, and (iii) a Lie-algebra parameterization $U_\theta = e^{iH(\theta)}$ (constructions after Kiani [2020]). Evaluation uses four datasets: a realizable custom dataset of 3-qubits, generated from a circuit that lies strictly within the hypothesis space of the VQPT ansatz, establishing a realizable setting to validate effectiveness of the approach in its own ansatz; random 6-qubit circuit consisting of only $\{R_x, R_y, R_z\}+\mathrm{CNOT}$ which is close to the space in which the PQC was trained on; a 6-qubit QFT, and a 7-qubit random circuit - under both noiseless and noisy conditions. All results are reported with the fidelity of Eq. (14) for consistency. As reported in Table 2, VQPT and Lie-based methods achieve near-perfect reconstruction on the custom dataset ($\approx$ 0.99 fidelity), as the target unitaries lie within their shared ansatz space, whereas the Givens-based approach attains comparatively lower fidelity. This clearly suggests the ansatz space of one need not lie in the ansatz space of others – this challenge gets exacerbated when noise is present. But all of them suffer marked degradation on the random, QFT and 7-qubit sets. By contrast, the Stiefel approach maintains high fidelity and similarity (> 0.97) across all datasets and noise levels. This clearly demonstrate that our method can do unconstrained approximation of any ansatz, even with noise, while retaining the unitarity.

Due to space constraints, our appendix contains additional experiments, such as impact of m noisy training pairs, convergence of our method in the quantum circuit space, and additional examples supporting the discussed results.

5 CONCLUSION

We introduced a geometric optimization framework for learning unitary transformations from noisy quantum data. By constraining weights to the Stiefel manifold with Cayley updates, the method preserves unitarity, ensures monotonic descent, and mitigates vanishing gradients in deep networks. Experiments show that deeper architectures converge faster and achieve higher fidelity, with stable convergence across noise levels and learning rates. This work establishes a principled link between geometric optimization and quantum gate learning, with future extensions toward general quantum operations and hardware implementation.

References

P.-A. Absil, R. Mahony, and R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2009. ISBN 9780691132983.

Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Lev S. Bishop, et al. Qiskit: An open-source framework for quantum computing. *Nature Quantum Information*, 8:1–5, 2022. doi: 10.1038/s41534-022-00519-6.

Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. In *International conference on machine learning*, pages 242–252. PMLR, 2019.

Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1120–1128, 2016.

María Barbero-Liñán and David Martín de Diego. Retraction maps: A seed of geometric integrators. *Foundations of Computational Mathematics*, 23:1335–1380, 2023. doi: 10.1007/s10208-022-09571-x. URL https://doi.org/10.1007/s10208-022-09571-x. Published online: 8 June 2022; Received: 1 June 2021 / Revised: 7 February 2022 / Accepted: 25 February 2022.

Mikhail Gennadievich Belov. Quantum channel learning. *arXiv preprint arXiv:2407.04406*, 2024. URL https://arxiv.org/abs/2407.04406.

Samuel Yen-Chi Chen, Chao-Han Huck Yang, Jun Qi, Pin-Yu Chen, Xiaoli Ma, and Hsi-Sheng Goan. Variational quantum circuits for deep reinforcement learning. *IEEE Access*, 2020. doi: 10.1109/ACCESS.2020.2968539.

Eric C Cyr, Mamikon A Gulian, Ravi G Patel, Mauro Perego, and Nathaniel A Trask. Robust training and initialization of deep neural networks: An adaptive basis viewpoint. In *Mathematical and Scientific Machine Learning*, pages 512–536. PMLR, 2020.

Alan Edelman, Tomás A. Arias, and Steven T. Smith. The geometry of algorithms with orthogonality constraints. *SIAM Journal on Matrix Analysis and Applications*, 20 (2):303–353, 1998.

Mark Embree, Joel A Henningsen, Jordan Jackson, and Ronald B Morgan. Polynomial approximation to the inverse of a large matrix. *SIAM Journal on Scientific Computing*, (0):S258–S284, 2025.

Yan Ge, Wu Wenjie, Chen Yuheng, et al. Quantum circuit synthesis and compilation optimization: Overview and prospects. *Preprint (arXiv)*, 2024. Provides a survey on translating algorithms into optimized gate sequences under NISQ constraints.

Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.

Akira Hirose. *Complex-Valued Neural Networks*. Springer, 2012. ISBN 9783642301394.

Essam H. Houssein, Zainab Abohashima, Mohamed Elhoseny, and Waleed M. Mohamed. Machine learning in the quantum realm: The state-of-the-art, challenges, and future vision. *Expert Systems with Applications*, 194: 116512, 2022. doi: 10.1016/j.eswa.2022.116512.

Yi-Ming Huang, Xiao-Yu Li, Yi-Xuan Zhu, Hang Lei, Qing-Sheng Zhu, and Shan Yang. Learning unitary transformation by quantum machine learning model. *Computers, Materials & Continua*, 68(1):789–803, 2021. doi: 10.32604/cmc.2021.016663. URL https://doi.org/10.32604/cmc.2021.016663.

Bobak Toussi Kiani. Quantum artificial intelligence: Learning unitary transformations. S.m. thesis, Massachusetts Institute of Technology, Cambridge, MA, USA, May 2020. URL https://hdl.handle.net/1721.1/127158.

Kelvin Koor, Yixian Qiu, Leong Chuan Kwek, and Patrick Rebentrost. A short tutorial on wirtinger calculus with applications in quantum information. *arXiv preprint arXiv:2312.04858*, 2023.

Kehelwala D.G̃. Maduranga, Kyle E. Helfrich, and Qiang Ye. Complex unitary recurrent neural networks using scaled cayley transform. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence*, AAAI'19, pages 4528–4535. Association for the Advancement of Artificial Intelligence, 2019. doi: 10.1609/aaai.v33i01.33014528. URL https://doi.org/10.1609/aaai.v33i01.33014528.

Iman Marvian and Seth Lloyd. Efficient quantum emulation of unknown unitaries. *PRX Quantum*, 6(3):030346, 2025. doi: 10.1103/PRXQuantum.6.030346. URL https://link.aps.org/doi/10.1103/PRXQuantum.6.030346.

Md Mobasshir Arshed Naved, Wenbo Xie, Wojciech Szpankowski, and Ananth Grama. Robust integrated learning and pauli noise mitigation for parameterized quantum circuits. In *Advances in Neural Information Processing Systems (NeurIPS 2025) Posters*, 2025. URL https://neurips.cc/virtual/2025/poster/115015. Poster: Robust Integrated Learning and Pauli Noise Mitigation for Parametrized Quantum Circuits.

Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2002.

Peter J. Schreier and Louis L. Scharf. Complex differential calculus (wirtinger calculus). In *Statistical Signal Processing of Complex-Valued Data*, pages 277–286. Cambridge University Press, 2010. doi: 10.1017/CBO9780511815911.014.

Ali Shaib, Mohamad H. Naim, Mohammed E. Fouda, Rouwaida Kanj, and Fadi Kurdahi. Efficient noise mitigation technique for quantum computing. *IEEE Access*, 11:14609–14620, 2023. doi: 10.1109/ACCESS.2023.3241234.

Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994. doi: 10.1109/SFCS.1994.365700.

Michael Spivak. *A Comprehensive Introduction to Differential Geometry*, volume 1–5. Publish or Perish, 1979.

Hemant D. Tagare. Notes on optimization on stiefel manifolds. Technical report, Yale University, 2011.

Bobak Toussi Kiani, Seth Lloyd, and Reevu Maity. Learning unitaries by gradient descent. *arXiv preprint arXiv:2001.11897*, 2020. URL `https://arxiv.org/abs/2001.11897`.

Scott Wisdom, Thomas Powers, John R. Hershey, Jonathan Le Roux, and Les Atlas. Full-capacity unitary recurrent neural networks. In *Advances in Neural Information Processing Systems*, pages 4880–4888, 2016.

Shichuan Xue, Yong Liu, Yang Wang, Pingyu Zhu, Chu Guo, and Junjie Wu. Variational quantum process tomography. *Physical Review A*, 105(3):032427, 2021. doi: 10.1103/PhysRevA.105.032427. URL `https://link.aps.org/doi/10.1103/PhysRevA.105.032427`.

Peng Zhang and Yuxiang Gao. Matrix multiplication on high-density multi-gpu architectures: theoretical and experimental investigations. In *International Conference on High Performance Computing*, pages 17–30. Springer, 2015.

M. Zomorodi, H. Amini, M. Abbaszadeh, J. Sohrabi, V. Salari, and P. Plawiak. Optimal quantum circuit design via unitary neural networks. arXiv preprint arXiv:2408.13211, 2024.

APPENDIX

The appendix contains the proof of statements in the main paper (Secs. A-C), space-time complexity analysis (Sec. D), additional experiments (Sec. E), and implementation details (Sec. F).

OVERALL WORKFLOW OF THE PROPOSED STIEFEL-MANIFOLD-BASED OPTIMIZATION FRAMEWORK

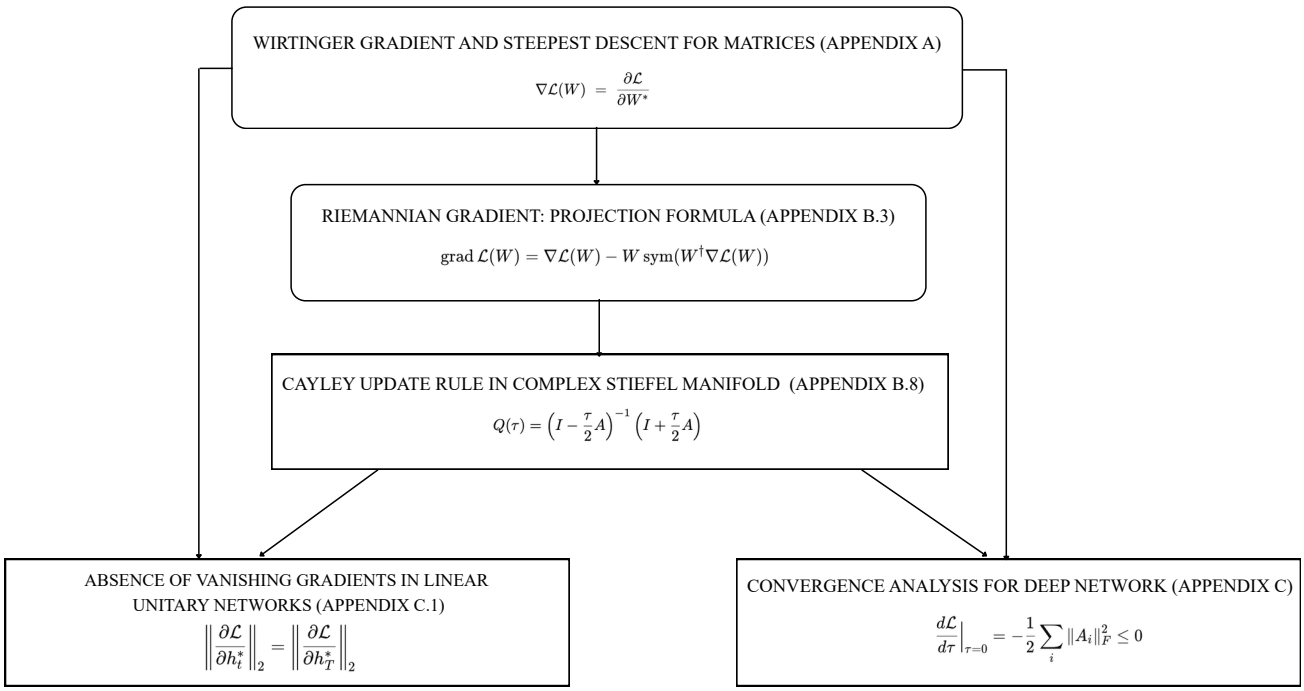


Figure 6: Our overall workflow: The pipeline consists of Wirtinger gradient computation, Riemannian gradient projection onto the tangent space, Cayley-based manifold update, and convergence analysis, with theoretical guarantees on gradient behavior and stability (see Secs. A-C)).

A WIRTINGER GRADIENT AND STEEPEST DESCENT FOR MATRICES IN $\in \mathbb{C}^{N\times N}$

A.1 PRELIMINARIES OF WIRTINGER CALCULUS

In complex function theory, we use $z = x + iy$ for complex numbers, where $z \in \mathbb{C}$ and $x, y \in \mathbb{R}$. Complex functions are denoted as $f : \mathbb{C} \to \mathbb{C}$, with $f(z) = u(x, y) + iv(x, y)$ and $u, v : \mathbb{R} \to \mathbb{R}$.

Definition A.1. A complex function f is differentiable (in complex domain) at z, with the derivative $f'(z)$, if the following limit exists

$$f'(z) = \lim_{h\to 0} \frac{f(z+h) - f(z)}{h}. \tag{17}$$

It is known that f is differentiable at point z if and only if u and v are differentiable, and the Cauchy Riemann equations hold at z:

$$\frac{\partial u}{\partial x} = \frac{\partial v}{\partial y} \qquad \frac{\partial u}{\partial y} = -\frac{\partial v}{\partial x}. \tag{18}$$

A loss function *cannot* be complex valued, as there exists no notion of orderedness (e.g., one cannot determine whether $2 + 3j$ is equal to, or greater than, or lesser than $3 + 2j$). If the loss function f is real-valued (i.e., $v = 0$) and bounded, the Cauchy-Riemann (CR) equations simplify to $\frac{\partial u}{\partial x} = \frac{\partial u}{\partial y} = 0$, implying that f must be constant – thereby losing the significance of a loss function. Wirtinger derivatives provide an alternative approach to complex derivatives.

Definition A.2. The Wirtinger derivatives operators for a complex valued function $f : \mathbb{C} \to \mathbb{C}$, assuming $u(x, y)$ and $v(x, y)$ are differentiable at x and y, are given by Koor et al. [2023]

$$\frac{\partial f}{\partial z} := \frac{1}{2}\left(\frac{\partial f}{\partial x} - i\frac{\partial f}{\partial y}\right), \tag{19}$$

$$\frac{\partial f}{\partial z^*} := \frac{1}{2}\left(\frac{\partial f}{\partial x} + i\frac{\partial f}{\partial y}\right), \tag{20}$$

where $\frac{\partial}{\partial x}$ and $\frac{\partial}{\partial y}$ operators are

$$\frac{\partial f}{\partial x} := \frac{\partial u}{\partial x} + i\frac{\partial v}{\partial x} \text{ and } \frac{\partial f}{\partial y} := \frac{\partial u}{\partial y} + i\frac{\partial v}{\partial y}. \tag{21}$$

The chain rule in Wirtinger calculus, which is pivotal to back propagation algorithm to train deep complex valued neural network, translates to Koor et al. [2023]

$$\frac{\partial \mathcal{L}(f)}{\partial z^*} = \frac{\partial f}{\partial z^*}\frac{\partial \mathcal{L}}{\partial f} + \frac{\partial f^*}{\partial z^*}\frac{\partial \mathcal{L}}{\partial f^*}. \tag{22}$$

A.2 PROOF OF PROPOSITION 3.1

Lemma A.3 (Conjugate Property of Wirtinger Gradient)**.** *For Wirtinger gradient, the following relation holds:*

$$\left(\frac{\partial f}{\partial z}\right)^* = \frac{\partial f}{\partial z^*}. \tag{23}$$

Proof. Consider the complex conjugate of the Wirtinger derivative of f at c. We have:

$$\left(\frac{\partial f}{\partial z}\right)^* = \frac{1}{2}\left(\frac{\partial u}{\partial x} + i\frac{\partial v}{\partial y} - i\left(\frac{\partial u}{\partial y} + i\frac{\partial v}{\partial y}\right)\right)^*, \tag{24}$$

$$= \frac{1}{2}\left(\frac{\partial u}{\partial x} + i\frac{\partial(-v)}{\partial x} + i\left(\frac{\partial u}{\partial y} + i\frac{\partial(-v)}{\partial y}\right)\right), \tag{25}$$

$$= \frac{\partial f^*}{\partial x} + i\frac{\partial f^*}{\partial y} = \frac{\partial f^*}{\partial z^*}. \tag{26}$$

Hence proved. One corollary is that, if f is a real valued function,

$$\left(\frac{\partial f}{\partial z}\right)^* = \frac{\partial f^*}{\partial z^*} = \frac{\partial f}{\partial z^*}, \qquad (\text{as } f^* = f, \text{ if } f \text{ is real}). \tag{27}$$

□

Lemma A.4 (Multivariate Wirtinger Derivative along a given direction)**.** *Consider a complex function $f : \mathbb{C}^N \to \mathbb{C}$ that takes n complex variables $z_1, z_2, \ldots, z_n$ and maps them to a complex number. The variables z_j can be expressed in terms of their real and imaginary parts: $z_j = x_j + iy_j$. The derivative of f along an infinitesimal $h_j = a_j + ib_j$ is given by:*

$$df = \sum_{j=1}^{n}\left(\frac{\partial f}{\partial z_j}h_j + \frac{\partial f}{\partial {z_j}^*}{h_j}^*\right), \tag{28}$$

where ${h_j}^*$ is its complex conjugate.:

Proof. The change in f, denoted by df, can be written as:

$$df = f(z_1 + h_1, z_2 + h_2, \ldots, z_n + h_n) - f(z_1, z_2, \ldots, z_n). \tag{29}$$

Let $u(x_1, y_1, \ldots, x_n, y_n)$ and $v(x_1, y_1, \ldots, x_n, y_n)$ represent the real and imaginary parts of f, respectively. Then, we can express the total derivative as:

$$\begin{aligned} du &= u(x_1 + a_1, y_1 + b_1, \ldots) - u(x_1, y_1, \ldots), \\ dv &= v(x_1 + a_1, y_1 + b_1, \ldots) - v(x_1, y_1, \ldots), \end{aligned} \tag{30}$$

where a_j and b_j are infinitesimal increments in the real and imaginary parts of z_j, respectively.

The change in f, denoted by df, can thus be written as:

$$df = \sum_{j=1}^{n} \left(\frac{\partial u}{\partial x_j} + i \frac{\partial v}{\partial x_j} \right) a_j + \left(\frac{\partial u}{\partial y_j} + i \frac{\partial v}{\partial y_j} \right) b_j. \tag{31}$$

Note that $a_j = \frac{h_j + h_j^*}{2}$ and $b_j = \frac{h_j - h_j^*}{2i}$.

We can simplify this further by defining the multivariate Wirtinger derivatives (extending Eq. (21)):

$$\frac{\partial f}{\partial z_j} := \frac{1}{2} \left(\frac{\partial f}{\partial x_j} - i \frac{\partial f}{\partial y_j} \right), \quad \frac{\partial f}{\partial z_j^*} := \frac{1}{2} \left(\frac{\partial f}{\partial x_j} + i \frac{\partial f}{\partial y_j} \right). \tag{32}$$

Using these Wirtinger derivatives and Eq. 21, we can express the derivative along h_j as:

$$df = \sum_{j=1}^{n} \left(\frac{\partial f}{\partial z_j} h_j + \frac{\partial f}{\partial {z_j}^*} {h_j}^* \right), \tag{33}$$

where $h_j = a_j + i b_j$ and ${h_j}^*$ is its complex conjugate.

This form shows that the derivative of a multivariate complex function along a direction can be decomposed into a sum of derivatives with respect to each complex variable and its conjugate. □

Proposition A.5 (Wirtinger Gradient is the Steepest Descent)**.** *For a real-valued loss function $\mathcal{L}(W)$ defined on the space of complex matrices $W_l \in \mathbb{C}^{N \times N}$, where $1 \leq l \leq L$, the direction of steepest descent is given by the Wirtinger gradient. For layer l at iteration k, this gradient is defined as:*

$$G_l^{(k)} = \frac{\partial \mathcal{L}}{\partial \left(W_l^{(k)} \right)^*} := \nabla \mathcal{L}(\mathcal{W}), \tag{34}$$

where * denotes the complex conjugate and $G_l^{(k)} \in \mathbb{C}^{N \times N}$. This gradient accounts for the dependence of $\mathcal{L}$ on all W_l, providing a valid descent direction ($-G_l^{(k)}$).

Proof. Let $\mathcal{L} : \mathbb{C}^{N \times N} \to \mathbb{R}$ be a real-valued function. Consider a point $W_l := (w_{ijl})_{i,j=1}^N \in \mathbb{C}^{N \times N}$ and a direction $H_l = (h_{ijl})_{i,j=1}^N \in \mathbb{C}^{N \times N}$ (note that we drop the iteration k for brevity). For a multi-layer network ($1 \leq l \leq L+1$), the derivative of $\mathcal{L}$ at W_l in the direction H_l for all l is given by:

$$d\mathcal{L} = \sum_{l=1}^{L+1} \sum_{i,j=1}^{N} \left(\frac{\partial \mathcal{L}}{\partial w_{ijl}} h_{ijl} + \frac{\partial \mathcal{L}}{\partial w_{ijl}^*} h_{ijl}^* \right), \qquad \text{(using Lemma } A.4\text{)}. \tag{35}$$

Since $\mathcal{L}$ is real-valued, we can write $G_l = \left(\frac{\partial \mathcal{L}}{\partial w_{ijl}^*} \right)_{i,j=1}^N$ and $G_l^* = \left(\frac{\partial \mathcal{L}}{\partial w_{ijl}} \right)_{i,j=1}^N$:

$$d\mathcal{L} = \sum_{l=1}^{L+1} 2Re\left(\sum_{i,j=1}^{N} \frac{\partial \mathcal{L}}{\partial w_{ij}} h_{ij}\right), \qquad \text{(using Lemma } A.3\text{)}. \tag{36}$$

The expression above can be written as:

$$d\mathcal{L} = \sum_{l=1}^{L+1} 2Re\left(\langle G_l, H_l\rangle_F\right), \tag{37}$$

where $\langle A, B\rangle_F = \mathrm{tr}(A^\dagger B) = \sum_{i,j=1}^{N} \overline{a_{ij}} b_{ij}$ is the Frobenius inner product.

We want to find the direction H_l that maximizes the directional derivative $d\mathcal{L}$ subject to a fixed step size, say $\|H_l\|_F = \epsilon$ for a small $\epsilon > 0$, where $\|H_l\|_F = \sqrt{\langle H_l, H_l\rangle_F}$ is the Frobenius norm.

By the Cauchy-Schwarz inequality for the Frobenius inner product, we have for each term in Eq. 37:

$$Re\left(\langle G_l, H_l\rangle_F\right) \leq |\langle G_l, H_l\rangle_F| \leq \|G_l\|_F \|H_l\|_F. \tag{38}$$

The maximum of $Re\left(\langle G_l, H_l\rangle_F\right)$ is achieved when H_l is in the same direction as G_l, i.e., when $H_l = \alpha G_l$ for some $\alpha > 0$. More precisely, the maximum value is $\|G_l\|_F \|H_l\|_F$ and occurs when H_l is a positive real multiple of G_l. Therefore, the steepest ascent direction at iteration k is along $G_l^{(k)}$.

Hence, the steepest descent direction (the direction that decreases $\mathcal{L}$ the most) at iteration k is the opposite: $-G_l^{(k)}$. □

B DERIVATION OF CAYLEY UPDATE RULE IN COMPLEX STIEFEL MANIFOLD

B.1 DERIVATION OUTLINE

The derivation of the Cayley update rule for multi-layer unitary networks follows a systematic geometric optimization framework. First, since the loss function $\mathcal{L} : \mathbb{C}^{N\times N} \to \mathbb{R}$ is real-valued, the standard Euclidean gradient does not exist; we instead compute the Wirtinger gradient $\nabla\mathcal{L}(\mathcal{W})$, negative of which provides the steepest descent direction in the ambient complex space (Appendix A). However, this direction generally points outside the Stiefel manifold. We therefore project it onto the tangent space $T_W\mathcal{M}$ (Appendix B.2) to obtain the Riemannian gradient $\mathrm{grad}\,\mathcal{L}(W)$ (Appendix B.3), negative of which provides the steepest descent direction (Appendix B.4) *within* the tangent space. To move along the descent direction while staying on the unitary manifold, we require a retraction map $R_W : T_W\mathcal{M} \to \mathcal{M}$ (Appendix B.6). The ideal retraction is given by the exponential map $R_W(\tau Z) = \exp(\tau A)W$ (Appendix B.7), but this is computationally expensive for large matrices. Instead, we use the Cayley transform as a first-order approximation to the exponential map (Appendix B.8), yielding the efficient update rule $W^{(k+1)} = \left(I + \frac{\lambda}{2}A^{(k)}\right)^{-1}\left(I - \frac{\lambda}{2}A^{(k)}\right)W^{(k)}$. For an L-layer network, we apply this update simultaneously to all layers. We prove that unitarity is preserved at every layer and that the overall loss decreases monotonically for sufficiently small step sizes. Finally, due to the unitary constraint, the network enjoys inherent protection against vanishing and exploding gradients, enabling stable training of deep architectures.

B.2 STIEFEL MANIFOLD AND ITS TANGENT SPACE

Definition B.1 (Complex Stiefel manifold)**.** Considering square matrices, the Stiefel manifold is defined as Tagare [2011]

$$\mathcal{M} = \mathcal{V}_N(\mathbb{C}^N) = \{W \in \mathbb{C}^{N\times N} : W^\dagger W = I\}. \tag{39}$$

This is precisely the set of $N \times N$ unitary matrices.

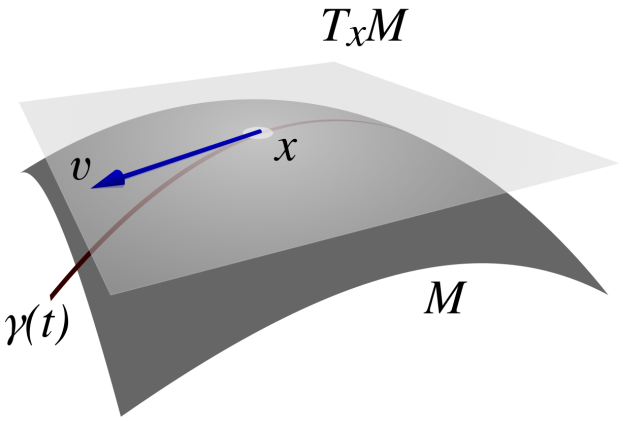


Figure 7: The tangent space $T_x\mathcal{M}$ and a tangent vector $v \in T_x\mathcal{M}$ associated with a curve passing through the point $x \in \mathcal{M}$.

B.2.1 Tangent Space of the Stiefel Manifold

The tangent space of a manifold is a generalization of tangent lines to curves in two-dimensional space and tangent planes to surfaces in three-dimensional space in higher dimensions. The tangent space at a point on a manifold is the vector space that contains every possible instantaneous direction you can move from that point while staying on the manifold.
One can attach to every point W of a differentiable manifold (A differentiable manifold is a space that is locally equivalent to $\mathbb{R}^n$ and equipped with a smooth structure that allows calculus to be performed consistently across the space) a tangent space—a vector space that intuitively contains the possible directions in which one can tangentially pass through W. The elements of the tangent space at W are called the tangent vectors at W.

Proposition B.2 (Tangent space)**.** *Suppose $\mathcal{M}$ be the complex Stiefel manifold of square matrices. The tangent space at $W \in \mathcal{M}$ is*

$$T_W\mathcal{M} = \{Z \in \mathbb{C}^{N\times N} : \ Z^\dagger W + W^\dagger Z = 0\}, \tag{40}$$

equivalently,

$$T_W\mathcal{M} = \{W\Omega : \ \Omega \in \mathbb{C}^{N\times N}, \Omega^\dagger = -\Omega\}. \tag{41}$$

This characterization implies that every tangent vector can be represented as the product of the current point W and a skew-Hermitian matrix Ω. This representation is particularly useful for constructing descent directions that preserve unitarity, as any update along $W\Omega$ remains on the manifold to first order.

Let $W_0 \in \mathcal{M}$ and let $t \mapsto W(t)$ be a curve in $\mathcal{M}$ through W_0 at $t = 0$, i.e., $W(0) = W_0$ and

$$W(t)^\dagger W(t) = I, \qquad \text{for all } t. \tag{42}$$

Differentiating (42) gives

$$W'(t)^\dagger W(t) + W(t)^\dagger W'(t) = 0. \tag{43}$$

Setting $t = 0$, $W(0) = W_0$, and $W'(0) = Z$, we obtain

$$Z^\dagger W_0 + W_0^\dagger Z = 0. \tag{44}$$

Hence the tangent space at W_0 is contained in

$$\{Z \in \mathbb{C}^{N\times N} : Z^\dagger W_0 + W_0^\dagger Z = 0\}. \tag{45}$$

In fact, this set exactly equals the tangent space $T_{W_0}\mathcal{M}$. Thus we can write,

$$T_W\mathcal{M} = \{Z \in \mathbb{C}^{N\times N} : Z^\dagger W + W^\dagger Z = 0\}. \tag{46}$$

We now show an alternative characterization of $T_W\mathcal{M}$. As W_0 is unitary, its columns form an orthonormal basis of $\mathbb{C}^N$. Any matrix $Z \in \mathbb{C}^{N\times N}$ can be written as $Z = W_0 A$ for some $A \in \mathbb{C}^{N\times N}$ (since W_0 is invertible). Substituting into (44) yields

$$(W_0 A)^\dagger W_0 + W_0^\dagger (W_0 A) = A^\dagger + A = 0, \tag{47}$$

so $A^\dagger = -A$, i.e., A is skew-Hermitian. Therefore,

$$T_W\mathcal{M} = \{W\Omega : \Omega \in \mathbb{C}^{N\times N},\ \Omega^\dagger = -\Omega\}. \tag{48}$$

Lemma B.3 (Normal space)**.** *The normal space $(T_W\mathcal{M})^\perp$ at W consists of matrices of the form WS, where S is a Hermitian $N \times N$ matrix:*

$$(T_W\mathcal{M})^\perp = \{WS : S \in \mathbb{C}^{N\times N},\ S^\dagger = S\}. \tag{49}$$

Proof. Let $Z \in T_W\mathcal{M}$ and $Z^\perp \in (T_W\mathcal{M})^\perp$. By the characterization of the tangent space, we can write

$$Z = W\Omega, \qquad Z^\perp = WS, \tag{50}$$

where $\Omega^\dagger = -\Omega$ (skew-Hermitian) and $S^\dagger = S$ (Hermitian).

The Riemannian metric on the Stiefel manifold is given by the real part of the Frobenius inner product:

$$\langle A, B\rangle = \operatorname{Re}\operatorname{tr}\big(A^\dagger B\big). \tag{51}$$

We compute the inner product between Z and $Z^\perp$:

$$\begin{aligned}
\langle Z, Z^\perp\rangle &= \operatorname{Re}\operatorname{tr}\big((W\Omega)^\dagger (WS)\big) && (52)\\
&= \operatorname{Re}\operatorname{tr}\big(\Omega^\dagger W^\dagger W S\big) && (53)\\
&= \operatorname{Re}\operatorname{tr}\big(\Omega^\dagger S\big) \quad \text{since } W^\dagger W = I. && (54)
\end{aligned}$$

Now, consider the complex number $\operatorname{tr}(\Omega^\dagger S)$. Its complex conjugate is

$$\begin{aligned}
\big(\operatorname{tr}(\Omega^\dagger S)\big)^* &= \operatorname{tr}\big((\Omega^\dagger S)^\dagger\big) = \operatorname{tr}\big(S^\dagger \Omega\big) && (55)\\
&= \operatorname{tr}\big(S\Omega\big) \quad \text{since } S^\dagger = S && (56)\\
&= \operatorname{tr}\big(\Omega S\big) \quad \text{(trace is cyclic)} && (57)\\
&= -\operatorname{tr}\big(\Omega^\dagger S\big) \quad \text{since } \Omega^\dagger = -\Omega. && (58)
\end{aligned}$$

Thus, $\big(\operatorname{tr}(\Omega^\dagger S)\big)^* = -\operatorname{tr}(\Omega^\dagger S)$, which implies that $\operatorname{tr}(\Omega^\dagger S)$ is purely imaginary. Consequently, its real part is zero:

$$\operatorname{Re}\operatorname{tr}\big(\Omega^\dagger S\big) = 0. \tag{59}$$

Therefore, $\langle Z, Z^\perp\rangle = 0$, showing that every matrix of the form WS with S Hermitian is orthogonal to the tangent space $T_W\mathcal{M}$. Hence, WS belongs to the normal space $(T_W\mathcal{M})^\perp$. □

B.3 RIEMANNIAN GRADIENT: PROJECTION FORMULA

Theorem B.4 (Riemannian Gradient on Stiefel Manifold)**.** *For a smooth function $\mathcal{L} : \mathbb{C}^{N\times N} \to \mathbb{R}$, the Riemannian gradient of the loss function ($\operatorname{grad}\mathcal{L}(W)$) is the orthogonal projection (P_W) of Wirtinger gradient of the loss function ($\nabla\mathcal{L}(W)$) on the tangent space ($T_W\mathcal{M}$) (Absil et al. [2009]) and it is given by :*

$$\operatorname{grad}\mathcal{L}(W) = P_W(\nabla\mathcal{L}(W)) = \nabla\mathcal{L}(W) - W\cdot\operatorname{sym}(W^\dagger\nabla\mathcal{L}(W)), \tag{60}$$

where $P_W : \mathbb{C}^{N\times N} \to T_W\mathcal{M}$ is the orthogonal projector and $\operatorname{sym}(A) = \frac{1}{2}(A + A^\dagger)$ and $\nabla\mathcal{L}(W)$ is the Wirtinger gradient.

Proof. We can write $\nabla\mathcal{L}(W) = G = P_W(G) + P_W^{\perp}(G)$ with $P_W(G) \in T_W\mathcal{M}$ and $P_W^{\perp}(G) \in (T_W\mathcal{M})^{\perp}$. By Lemma 1.1, there exists a Hermitian matrix S such that $P_W^{\perp}(G) = WS$. Hence

$$P_W(G) = G - WS. \tag{61}$$

To determine S, we use that $P_W(G)$ must be tangent, i.e., $P_W(G) \in T_W\mathcal{M}$, therefore it should satisfy (44):

$$W^\dagger P_W(G) + P_W(G)^\dagger W = 0. \tag{62}$$

By substituting (61) in (62) and using $S^\dagger = S$ gives

$$\begin{aligned} 2S &= W^\dagger G + G^\dagger W, \\ \implies S &= \tfrac{1}{2}\big(W^\dagger G + G^\dagger W\big) = \operatorname{sym}\big(W^\dagger G\big). \end{aligned} \tag{63}$$

Thus

$$P_W(G) = G - W\operatorname{sym}\big(W^\dagger G\big) \tag{64}$$

$$\implies P_W(\nabla\mathcal{L}(W)) = \nabla\mathcal{L}(W) - W\operatorname{sym}\big(W^\dagger \nabla\mathcal{L}(W)\big). \tag{65}$$

□

B.4 STEEPEST DESCENT IN THE TANGENT SPACE: RIEMANNIAN GRADIENT

Theorem B.5. *Let $\mathcal{M}$ be a Riemannian manifold embedded in Euclidean space, $\mathcal{L} : \mathcal{M} \to \mathbb{R}$ a smooth function, $\nabla\mathcal{L}(W) = G$ the Wirtinger gradient, and $\operatorname{grad}\mathcal{L}(W)$ the Riemannian gradient. Let P_W be the orthogonal projection onto the tangent space $T_W\mathcal{M}$. Then:*

1. *$\langle G, Z\rangle = \langle \operatorname{grad}\mathcal{L}(W), Z\rangle$ for all $Z \in T_W\mathcal{M}$ where $\langle\cdot,\cdot\rangle$ is Frobenius inner product.*
2. *The steepest descent direction property in the tangent space $T_W\mathcal{M}$ is maintained with Riemannian gradient.*

Proof. Let $P_W : \mathbb{C}^n \to T_W\mathcal{M}$ be the orthogonal projection operator onto the tangent space.

By the properties of orthogonal projection, any vector $G \in \mathbb{C}^{N\times N}$ can be uniquely decomposed as:

$$G = P_W(G) + (G - P_W(G)), \tag{66}$$

where $P_W(G) \in T_W\mathcal{M}$ is the projection onto the tangent space, and $G - P_W(G) \perp T_W\mathcal{M}$ is the normal component.

For any tangent vector $Z \in T_W\mathcal{M}$, we compute the inner product:

$$\langle G, Z\rangle = \langle P_W(G) + (G - P_W(G)), Z\rangle \tag{67}$$

$$= \langle P_W(G), Z\rangle + \langle G - P_W(G), Z\rangle. \tag{68}$$

Since $G - P_W(G)$ is orthogonal to the tangent space and $Z \in T_W\mathcal{M}$, we have:

$$\langle G - P_W(G), Z\rangle = 0. \tag{69}$$

Therefore:

$$\langle G, Z\rangle = \langle P_W(G), Z\rangle \quad \forall Z \in T_W\mathcal{M}. \tag{70}$$

As we know the directional derivative of $\mathcal{L}$ at W in the direction $Z \in T_W\mathcal{M}$:

$$D\mathcal{L}(W)[Z] = \langle G, Z\rangle. \tag{71}$$

The directional derivative remains the same for riemannain gradient because:

$$\begin{aligned} D\mathcal{L}(W)[Z] &= \langle G, Z\rangle \\ &= \langle P_W(G), Z\rangle + \langle G - P_W(G), Z\rangle \\ &= \langle P_W(G), Z\rangle + 0 \\ &= \langle \operatorname{grad}\mathcal{L}(W), Z\rangle. \end{aligned}$$

Therefore:

$$D\mathcal{L}(W)[Z] = \langle G, Z\rangle = \langle \operatorname{grad}\mathcal{L}(W), Z\rangle \quad \forall Z \in T_W\mathcal{M}. \tag{72}$$

The steepest descent direction on the manifold is found by:

$$\min_{Z\in T_W\mathcal{M}, \|Z\|=1} D\mathcal{L}(W)[Z] = \min_{Z\in T_W\mathcal{M}, \|Z\|=1} \langle G, Z\rangle. \tag{73}$$

Using the orthogonal projection property:

$$\begin{aligned} \min_{Z\in T_W\mathcal{M},\, \|Z\|=1} \langle G, Z\rangle &= \min_{Z\in T_W\mathcal{M},\, \|Z\|=1} \langle P_W(G), Z\rangle \\ &= \min_{Z\in T_W\mathcal{M},\, \|Z\|=1} \langle \operatorname{grad}\, \mathcal{L}(W), Z\rangle. \end{aligned} \tag{74}$$

By Cauchy-Schwarz, the minimum occurs when:

$$Z^* = -\frac{\operatorname{grad}\mathcal{L}(W)}{\|\operatorname{grad}\mathcal{L}(W)\|}. \tag{75}$$

This proves that the steepest descent direction property is maintained: the direction of fastest decrease on the tangent space $T_W\mathcal{M}$ is given by the Riemannian gradient, which is exactly the orthogonal projection of the Wirtinger gradient onto the tangent space. □

Corollary B.6. *For any embedded Riemannian manifold, the Riemannian optimization preserves the geometric interpretation of steepest descent while respecting the manifold constraints.*

B.5 EQUIVALENT REPRESENTATION OF THE RIEMANNIAN GRADIENT: $\operatorname{grad}\mathcal{L} = AW$

Remark B.7. The Riemannian gradient on certain matrix manifolds (such as the Stiefel manifold) admits a particularly elegant representation:

$$\operatorname{grad}\mathcal{L}(W) = AW, \tag{76}$$

where $A = \frac{1}{2}\left((\nabla\mathcal{L}(W))W^\dagger - W\nabla(\mathcal{L}(W))^\dagger\right)$ is a skew-Hermitian matrix ($A^\dagger = -A$).

Proof. From Theorem B.4, we know that the Riemannian gradient $(\operatorname{grad}\mathcal{L}(W))$ is given by

$$\operatorname{grad}\mathcal{L}(W) = \nabla\mathcal{L}(W) - W\,\operatorname{sym}\big(W^\dagger\nabla\mathcal{L}(W)\big), \tag{77}$$

where $\operatorname{sym}(X) = \frac{1}{2}(X + X^\dagger)$. Expanding the symmetrization yields

$$\operatorname{grad}\mathcal{L}(W) = \nabla\mathcal{L}(W) - W\cdot\frac{1}{2}\big(W^\dagger\nabla\mathcal{L}(W) + \nabla\mathcal{L}(W)^\dagger W\big) \tag{78}$$

$$= \nabla\mathcal{L}(W) - \tfrac{1}{2}\,WW^\dagger\nabla\mathcal{L}(W) - \tfrac{1}{2}\,W\nabla\mathcal{L}(W)^\dagger W. \tag{79}$$

Using $W^\dagger W = I$ and also (consequently) $WW^\dagger = I$ for square unitary W, the middle term simplifies and we obtain

$$\operatorname{grad}\mathcal{L}(W) = \nabla\mathcal{L}(W) - \tfrac{1}{2}\nabla\mathcal{L}(W) - \tfrac{1}{2}\,W\nabla\mathcal{L}(W)^\dagger W \tag{80}$$

$$= \tfrac{1}{2}\nabla\mathcal{L}(W) - \tfrac{1}{2}\,W\nabla\mathcal{L}(W)^\dagger W. \tag{81}$$

Factor W on the right by multiplying and dividing appropriate factors by $W^\dagger W = I$:

$$\operatorname{grad} \mathcal{L}(W) = \tfrac{1}{2}\big(\nabla \mathcal{L}(W) W^\dagger - W \nabla \mathcal{L}(W)^\dagger\big) W.$$

Thus defining

$$A \;:=\; \frac{1}{2}\big(\nabla \mathcal{L}(W) W^\dagger - W \nabla \mathcal{L}(W)^\dagger\big), \tag{82}$$

we have the desired representation

$$\operatorname{grad} \mathcal{L}(W) = AW, \tag{83}$$

As a remark, A is skew-Hermitian:

$$A^\dagger = \frac{1}{2}\big(W \nabla \mathcal{L}(W)^\dagger - \nabla \mathcal{L}(W) W^\dagger\big) = -A, \tag{84}$$

so AW indeed satisfies the tangent-space condition $W^\dagger(AW) + (AW)^\dagger W = 0$. □

B.6 RETRACTION FROM TANGENT SPACE TO STIEFEL MANIFOLD

Given a point $W \in \mathcal{M}$ and a tangent vector $Z \in T_W\mathcal{M}$, we often need to move away from W along the direction Z while remaining on the manifold: this is the basic operation of a gradient descent algorithm, and of essentially all optimization algorithms on manifolds. So, we have to map back to the manifold as we need to stay within the manifold while moving in that desired direction. This method of mapping back to the manifold from the tangent space is known as Retraction. We can achieve this by following any smooth curve c on $\mathcal{M}$ such that $c(0) = W$ and $c'(0) = Z$, but of course there exist many such curves.

A *retraction* picks a particular curve for each possible $(W, Z) \in T_W\mathcal{M}$. Furthermore, this choice of curve depends on (W, Z).

Definition B.8. A **retraction** at an arbitrary point W on a manifold $\mathcal{M}$ is a smooth map

$$R : T_W\mathcal{M} \to \mathcal{M}, \quad (W, Z) \mapsto R_W(Z) \tag{85}$$

such that each curve $c(t) = R_W(tZ)$ satisfies $c(0) = W$ and $c'(0) = Z$.

Now we have the descent direction from the Riemannian gradient and we need to find the retraction curve or update curve (W) which simultaneously stays in the manifold and moves towards the descent direction. We use ODE flow to obtain the retraction curve in the descent direction.

B.7 ODE FLOW AND EXPONENTIAL RETRACTION

The ideal retraction for the Stiefel manifold comes from the solution of a specific ordinary differential equation (Spivak [1979]): Here we want to get the retraction curve for $W \in \mathcal{M}$

Proposition B.9. *Let $t \mapsto W(t)$ be a differentiable curve in Stiefel manifold $\mathcal{M}$. If A is a skew-Hermitian and $\frac{dW}{dt} = AW(t)$ with initial condition $W(0) \in \mathcal{M}$, then $W(t)$ is also in $\mathcal{M}$ and the ideal retraction is given by*

$$R_W(tZ) = \exp(tA) W(0), \tag{86}$$

where $Z = AW(0) \in T_W\mathcal{M}$.

Proof. As we want to show that $W(t)$ remains on the Stiefel manifold, let $Y(t) = W(t)^\dagger W(t)$, so we need to show that $Y(t) = W(t)^\dagger W(t)$ is constant i.e., I.

$$\frac{dY}{dt} = \frac{dW^\dagger}{dt} W + W^\dagger \frac{dW}{dt} \tag{87}$$

$$= (AW)^\dagger W + W^\dagger (AW) \tag{88}$$

$$= W^\dagger (A^\dagger + A) W = 0, \qquad (\text{ since } A^\dagger = -A). \tag{89}$$

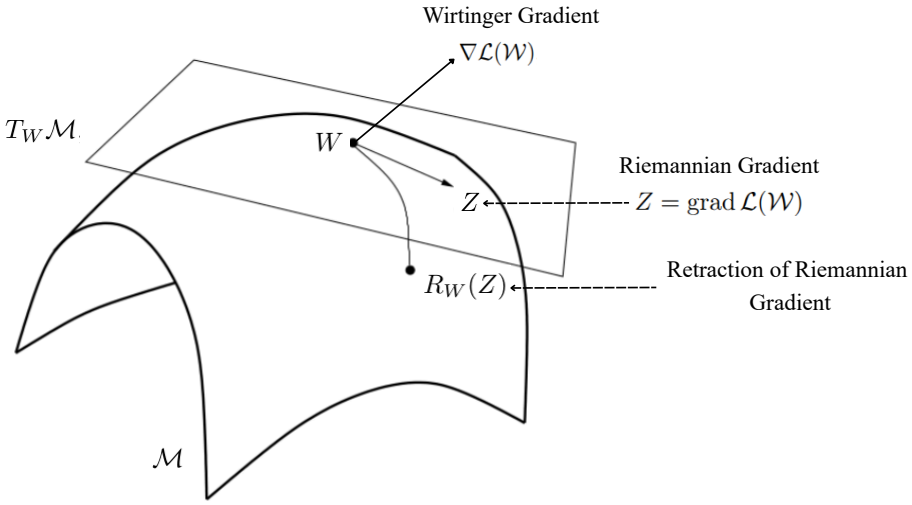


Figure 8: Illustration of a retraction on a manifold $\mathcal{M}$. Given a point $W \in \mathcal{M}$ and a tangent vector $Z \in T_W\mathcal{M}$, the retraction $R_W(Z)$ maps v back onto the manifold along a smooth curve.

Thus $Y(t)$ is constant, and since $W(0) \in \mathcal{M}$, we have $Y(t) = I$ for all t. Therefore we proved that the ODE, $\frac{dW}{dt} = AW(t)$ gives the ideal retraction curve for the Stiefel manifold. □

The solution to this ODE is given by the matrix exponential:

$$W(t) = \exp(tA)W(0). \tag{90}$$

So, for the curve $c(t) = W(t)$ to be an ideal retraction curve it should satisfy $c(0) = W(0)$ and $c'(0) = Z$, where $Z \in T_W\mathcal{M}$ is some direction. Since matrix exponential of a skew Hermitian matrix is unitary, $W(t)$ always stays in the Stiefel manifold. We can show easily other properties of retraction:

$$c'(t) = W'(t) = Ae^{tA}W(0), \tag{91}$$

$$\implies c'(0) = W'(0) = AW(0), \tag{92}$$

Since, $AW(0) = Z$, so we have the ideal retraction expression:

$$c(t) = R_W(tZ) = W(t) = \exp(tA)W(0), \tag{93}$$

However, computing matrix exponentials is computationally expensive for large matrices.

B.8 CAYLEY TRANSFORM: UNITARITY AND APPROXIMATION

As computing matrix exponential is expensive, therefore in order to get the retraction curve we try to approximate the matrix exponential. For a skew-Hermitian matrix $A \in \mathbb{C}^{N\times N}$ (i.e., $A^\dagger = -A$), the matrix exponential

$$\exp(\tau A) = I + \tau A + \frac{\tau^2}{2}A^2 + \frac{\tau^3}{6}A^3 + \cdots \tag{94}$$

maps A to a unitary matrix. However, computing the full exponential is costly for large n. The Cayley transform provides an efficient unitary approximation that agrees with the exponential up to second order.

Lemma B.10 (Cayley transform approximation)**.** *Let A be skew-Hermitian. Define the Cayley transform*

$$Q(\tau) = \left(I - \frac{\tau}{2}A\right)^{-1}\left(I + \frac{\tau}{2}A\right). \tag{95}$$

Then for small $\tau > 0$,

$$Q(\tau) = \exp(\tau A) + \mathcal{O}(\tau^3). \tag{96}$$

In particular, $Q(0) = I$ and $\frac{d}{d\tau} Q(\tau)\big|_{\tau=0} = A$, so $Q(\tau)$ is a first-order retraction on the unitary group.

Proof. First, note that the inverse $\left(I - \frac{\tau}{2}A\right)^{-1}$ exists for sufficiently small τ and can be expanded as a geometric series:

$$\left(I - \frac{\tau}{2}A\right)^{-1} = I + \frac{\tau}{2}A + \left(\frac{\tau}{2}\right)^2 A^2 + \left(\frac{\tau}{2}\right)^3 A^3 + \cdots . \tag{97}$$

Multiplying by $\left(I + \frac{\tau}{2}A\right)$,

$$\begin{aligned} Q(\tau) &= \left[I + \frac{\tau}{2}A + \left(\frac{\tau}{2}\right)^2 A^2 + \left(\frac{\tau}{2}\right)^3 A^3 + \cdots\right]\left(I + \frac{\tau}{2}A\right) \\ &= I + \frac{\tau}{2}A + \left(\frac{\tau}{2}\right)^2 A^2 + \left(\frac{\tau}{2}\right)^3 A^3 + \cdots \\ &\quad + \frac{\tau}{2}A + \left(\frac{\tau}{2}\right)^2 A^2 + \left(\frac{\tau}{2}\right)^3 A^3 + \cdots \\ &= I + \tau A + \frac{\tau^2}{2}A^2 + \frac{\tau^3}{4}A^3 + \cdots . \end{aligned}$$

On the other hand, the Taylor expansion of the exponential is

$$\exp(\tau A) = I + \tau A + \frac{\tau^2}{2}A^2 + \frac{\tau^3}{6}A^3 + \cdots . \tag{98}$$

Comparing the two series, the terms up to $\mathcal{O}(\tau^2)$ coincide; the difference appears at order τ^3 and higher. Hence

$$Q(\tau) = \exp(\tau A) + \mathcal{O}(\tau^3). \tag{99}$$

Moreover,

$$Q(0) = I, \qquad \frac{d}{d\tau} Q(\tau)\bigg|_{\tau=0} = A, \tag{100}$$

and also

$$exp(\tau A)|_{\tau=0} = I, \qquad \frac{d}{d\tau} exp(\tau A)\bigg|_{\tau=0} = A, \tag{101}$$

which are precisely the conditions required for a retraction on the unitary manifold. Thus the Cayley transform provides a valid, computationally efficient retraction that approximates the exponential map. □

We use the Cayley transform (Edelman et al. [1998]) as an efficient first-order approximation to the matrix exponential (Barbero-Liñán and Martín de Diego [2023]):

Lemma B.11. *If A is skew-Hermitian, then $Q = (I - \frac{\tau}{2}A)^{-1}(I + \frac{\tau}{2}A)$ is unitary.*

Proof.

$$\begin{aligned} Q^\dagger Q &= (I + \tfrac{\tau}{2}A)^\dagger (I - \tfrac{\tau}{2}A)^{-\dagger}(I - \tfrac{\tau}{2}A)^{-1}(I + \tfrac{\tau}{2}A) \\ &= (I + \tfrac{\tau}{2}A^\dagger)(I - \tfrac{\tau}{2}A^\dagger)^{-1}(I - \tfrac{\tau}{2}A)^{-1}(I + \tfrac{\tau}{2}A) \\ &= (I - \tfrac{\tau}{2}A)((I - \tfrac{\tau}{2}A)(I + \tfrac{\tau}{2}A))^{-1}(I + \tfrac{\tau}{2}A) \\ &= (I - \tfrac{\tau}{2}A)((I - \tfrac{\tau}{2}A) + \tfrac{\tau}{2}A(I - \tfrac{\tau}{2}A))^{-1}(I + \tfrac{\tau}{2}A) \\ &= (I - \tfrac{\tau}{2}A)((I + \tfrac{\tau}{2}A)(I - \tfrac{\tau}{2}A))^{-1}(I + \tfrac{\tau}{2}A) = I, \end{aligned}$$

where we used $A^\dagger = -A$. □

Now, our ideal retraction curve was $R_W(\tau Z) = \exp(\tau A)W$, here we replace the exponential with the Cayley transformation as following:

Definition B.12 (Cayley Retraction)**.** The Cayley retraction on the complex Stiefel manifold is defined as:

$$R_W(\tau Z) = \left(I - \frac{\tau}{2}A\right)^{-1}\left(I + \frac{\tau}{2}A\right)W, \tag{102}$$

where Z should be the descent direction.

Remark B.13. The descent direction is given by $Z = -\operatorname{grad}\mathcal{L}(W) = -AW$, by substituting the value of Z in the retraction curve we get the update law as following:

$$R_W(-\tau AW) = \left(I + \frac{\tau}{2}A\right)^{-1}\left(I - \frac{\tau}{2}A\right)W, \tag{103}$$

or we can write

$$W^{(k+1)} = R_W(-\tau AW^{(k)}) = \left(I + \frac{\tau}{2}A\right)^{-1}\left(I - \frac{\tau}{2}A\right)W^{(k)}. \tag{104}$$

This is the weight update law on the Stiefel manifold.

Proposition B.14. *The Cayley retraction $R_W(\tau Z)$ is a valid retraction as they satisfy the following conditions:*

1. $R_W(0) = W$.
2. $\frac{d}{d\tau}R_W(\tau Z)\big|_{\tau=0} = Z = -AW$.

Proof. 1. For $\tau = 0$, we have:

$$R_W(0) = (I)^{-1}(I)W = W, \tag{105}$$

which shows it follows condition (1).

2. Next to show condition 2, we differentiate $R_W(\tau Z) = \left(I + \frac{\tau}{2}A\right)^{-1}\left(I - \frac{\tau}{2}A\right)W$ where $Z = -AW$, with respect to τ. Now we know that if an arbitrary matrix $M(\tau)$ is invertible and differentiable, then

$$\frac{d}{d\tau}M(\tau)^{-1} = -M(\tau)^{-1}\frac{d}{d\tau}M(\tau)M(\tau)^{-1}. \tag{106}$$

Using this result, differentiating at $\tau = 0$ we obtain:

$$\frac{d}{d\tau}R_W(\tau Z)\big|_{\tau=0} = -\frac{1}{2}A \cdot W - \frac{1}{2}A \cdot W = -AW = Z. \tag{107}$$

□

Therefore we showed that this retraction is directed towards the steepest descent direction as $\frac{d}{d\tau}R_W(\tau Z)\big|_{\tau=0} = -\operatorname{grad}\mathcal{L}(W)$ and the retraction $R_W(\tau Z)$ is a valid retraction.

C CONVERGENCE ANALYSIS FOR DEEP NETWORK

Consider the composite loss function $\mathcal{L}(\tau) = \mathcal{L}(W_1(\tau), \ldots, W_{L+1}(\tau))$ where each $W_i(\tau)$ follows the retraction curve:

$$W_i(\tau) = \left(I + \frac{\tau}{2}A_i\right)^{-1}\left(I - \frac{\tau}{2}A_i\right)W_i \tag{108}$$

with A_i evaluated at the current weight matrix. To prove $\mathcal{L}$ to be a monotonically decreasing function, it is sufficient to show that the directional derivative of $\mathcal{L}$ at W in the direction of retraction is non-positive (Eq. 37). Now in the algorithm this ensures that if we choose small enough τ, this leads to decrease in $\mathcal{L}$.

For brevity, we denote the above condition as $\frac{d\mathcal{L}}{d\tau}\Big|_{\tau=0} \leq 0$. Therefore,

$$\frac{d\mathcal{L}}{d\tau}\Big|_{\tau=0} = \sum_{i=1}^{L+1} \mathrm{Re}\left\langle \nabla\mathcal{L}(W_i), \frac{dW_i}{d\tau}\Big|_{\tau=0} \right\rangle, \qquad \text{(using Eq. 37)}. \tag{109}$$

where $\langle A, B\rangle = \mathrm{tr}(A^\dagger B)$. From the retraction properties:

$$\frac{dW_i}{d\tau}(0) = -A_i W_i = -\operatorname{grad}_{W_i} \mathcal{L}. \tag{110}$$

Thus:

$$\begin{aligned}
\frac{d\mathcal{L}}{d\tau}\Big|_{\tau=0} &= -\sum_{i=1}^{L+1} \mathrm{Re}\left\langle \nabla\mathcal{L}(W_i), \operatorname{grad}_{W_i} \mathcal{L}\right\rangle && (111)\\
&= -\sum_{i=1}^{L+1} \mathrm{Re}\left\langle \nabla\mathcal{L}(W_i), A_i W_i\right\rangle && (112)\\
&= -\sum_{i=1}^{L+1} \mathrm{Re}\left[\mathrm{tr}\left((\nabla\mathcal{L}(W_i))^\dagger A_i W_i\right)\right]. && (113)
\end{aligned}$$

Substituting $A_i = (\nabla\mathcal{L}(W_i))W_i^\dagger - W_i(\nabla\mathcal{L}(W_i))^\dagger$:

$$\begin{aligned}
\frac{d\mathcal{L}}{d\tau}\Big|_{\tau=0} &= -\sum_{i=1}^{L+1} \mathrm{Re}\Big[\mathrm{tr}\Big((\nabla\mathcal{L}(W_i))^\dagger\big((\nabla\mathcal{L}(W_i))W_i^\dagger - W_i(\nabla\mathcal{L}(W_i))^\dagger\big)W_i\Big)\Big] && (114)\\
&= -\sum_{i=1}^{L+1} \mathrm{Re}\big[\mathrm{tr}\big((\nabla\mathcal{L}(W_i))^\dagger(\nabla\mathcal{L}(W_i)) - (\nabla\mathcal{L}(W_i))^\dagger W_i(\nabla\mathcal{L}(W_i))^\dagger W_i\big)\big] && (115)\\
&= -\sum_{i=1}^{L+1} \Big(\|\nabla\mathcal{L}(W_i)\|_F^2 - \mathrm{Re}\Big[\mathrm{tr}\Big(\big((\nabla\mathcal{L}(W_i))^\dagger W_i\big)^2\Big)\Big]\Big). && (116)
\end{aligned}$$

Let $B_i = (\nabla\mathcal{L}(W_i))^\dagger W_i$, where W_i is unitary and $\nabla\mathcal{L}(W_i)$ is the Wirtinger gradient. Therefore we have

$$\frac{d\mathcal{L}}{d\tau}\Big|_{\tau=0} = -\sum_{i=1}^{L+1} \left(\|\nabla\mathcal{L}(W_i)\|_F^2 - \mathrm{Re}\left[\mathrm{tr}\left(B_i^2\right)\right].\right) \tag{117}$$

Lemma C.1. *We can show that*

$$\mathrm{Re}[\mathrm{tr}(B_i^2)] \leq |\,\mathrm{tr}(B_i^2)| \leq \|B_i\|_F^2 = \|\nabla\mathcal{L}(W_i)\|_F^2. \tag{118}$$

Proof. We prove this using three parts.

1. **First inequality:** $\mathrm{Re}[\mathrm{tr}(B_i^2)] \leq |\,\mathrm{tr}(B_i^2)|$. This follows directly from the fact that for any complex number z, the real part satisfies $\mathrm{Re}(z) \leq |z|$.
2. **Second inequality:** $|\,\mathrm{tr}(B_i^2)| \leq \|B_i\|_F^2$. Let $\lambda_1, \ldots, \lambda_n$ be the eigenvalues of B_i. Then

$$|\,\mathrm{tr}(B_i^2)| = \left|\sum_{j=1}^{n} \lambda_j^2\right| \leq \sum_{j=1}^{n} |\lambda_j|^2. \tag{119}$$

For any matrix, the sum of squares of the moduli of its eigenvalues is bounded by the sum of squares of its singular values, which equals the squared Frobenius norm:

$$\sum_{j=1}^{n} |\lambda_j|^2 \leq \sum_{j=1}^{n} \sigma_j^2 = \|B_i\|_F^2. \tag{120}$$

where σ_j are the singular values of B_i. Alternatively, using the Cauchy–Schwarz inequality for complex matrices for the Frobenius inner product,

$$|\operatorname{tr}(B_i^2)| = |\langle B_i, B_i^\top \rangle| \leq \|B_i\|_F \|B_i^\top\|_F = \|B_i\|_F^2. \tag{121}$$

since the Frobenius norm is invariant under transpose.

3. **Equality:** $\|B_i\|_F^2 = \|\nabla_{W_i}\mathcal{L}\|_F^2$. Because W_i is unitary ($W_i^\dagger W_i = I$), we have

$$\|B_i\|_F^2 = \operatorname{tr}\big(B_i^\dagger B_i\big) = \operatorname{tr}\big(W_i^\dagger (\nabla_{W_i}\mathcal{L})(\nabla\mathcal{L}(W_i))^\dagger W_i\big) \tag{122}$$

$$= \operatorname{tr}\big((\nabla\mathcal{L}(W_i))(\nabla\mathcal{L}(W_i))^\dagger\big) = \|\nabla\mathcal{L}(W_i)\|_F^2. \tag{123}$$

Combining these results, we obtain

$$\operatorname{Re}[\operatorname{tr}(B_i^2)] \leq |\operatorname{tr}(B_i^2)| \leq \|B_i\|_F^2 = \|\nabla\mathcal{L}(W_i)\|_F^2.$$

Therefore,

$$\frac{d\mathcal{L}}{d\tau}\Big|_{\tau=0} \leq -\sum_{i=1}^{L+1} \big(\|\nabla\mathcal{L}(W_i)\|_F^2 - \|\nabla\mathcal{L}(W_i)\|_F^2\big) = 0. \tag{124}$$

Thus we proved that $\mathcal{L}(W_1, W_2, ..., W_L)$ is a monotonically decreasing function when τ is very small.

□

C.1 ABSENCE OF VANISHING GRADIENTS IN LINEAR UNITARY NETWORKS

Given the Wirtinger gradient $G_l^{(k)}$ and the current unitary weight matrix $W_l^{(k)}$, a descent direction on the complex Stiefel manifold is generated by the skew-Hermitian matrix:

$$A_l^{(k)} = G_l^{(k)}(W_l^{(k)})^\dagger - W_l^{(k)}(G_l^{(k)})^\dagger. \tag{125}$$

The unitary weight matrix at each layer l is then updated using a Cayley transform:

$$W_l^{(k+1)} = \left(I + \frac{\lambda}{2}A_l^{(k)}\right)^{-1}\left(I - \frac{\lambda}{2}A_l^{(k)}\right)W_l^{(k)}, \tag{126}$$

where $\lambda > 0$ is the learning rate.

Vanishing (or exploding) gradient problem occurs when the updates to the weights shrink (or enlarge) exponentially with network depth. From Eq. (126), this occurs when $G_l^{(k)}$ shrinks (or enlarge) in the initial layers. Consider a deep network with T hidden layers, with initial layer $t(\ll T)$ and $\mathcal{L}$ is the objective we are trying to minimize, then the vanishing (or exploding) gradient problem refers to

$$G_t^{(k)} = \frac{\partial\mathcal{L}}{\partial\left(W_t^{(k)}\right)^*} \to 0 \text{ (or } \infty), \qquad \text{for } t \ll T. \tag{127}$$

Let h_t be the output vector for hidden layer t, using the chain rule of Wirtinger gradient gives (Eq. (22)) and ignoring the iteration k for brevity,

$$\frac{\partial\mathcal{L}}{\partial W_t^*} = \frac{\partial h_t^*}{\partial W_t^*}\frac{\partial\mathcal{L}}{\partial h_t^*} + \frac{\partial h_t}{\partial W_t^*}\frac{\partial\mathcal{L}}{\partial h_t}. \tag{128}$$

Clearly, the influence of depth as the number of layers, T, grows ($t \ll T$) is factored in the terms $\frac{\partial\mathcal{L}}{\partial h_t}$ and $\frac{\partial\mathcal{L}}{\partial h_t^*}$. To show that the vanishing (or exploding) gradient problem depth does *not* occur, it suffices to show that the above terms does not shrink (or enlarge) with depth T, which we prove next.

Proposition C.2. *For a deep unitary network optimized using Wirtinger gradient, $\frac{\partial \mathcal{L}}{\partial {h_t}^*}$ remains constant irrespective of t for arbitrary high values of T. Specifically,*

$$\left\| \frac{\partial \mathcal{L}}{\partial {h_t}^*} \right\|_2 = \left\| \frac{\partial \mathcal{L}}{\partial {h_T}^*} \right\|_2 \qquad T \gg t, \tag{129}$$

$$\left\| \frac{\partial \mathcal{L}}{\partial h_t} \right\|_2 = \left\| \frac{\partial \mathcal{L}}{\partial h_T} \right\|_2 \qquad T \gg t. \tag{130}$$

Proof. Using the chain rule of Wirtinger calculus (Eq. (22)) and Lemma A.3, we get

$$\frac{\partial \mathcal{L}}{\partial {h_t}^*} = \frac{\partial h_T}{\partial {h_t}^*} \frac{\partial \mathcal{L}}{\partial h_T} + \left(\frac{\partial h_T}{\partial h_t} \right)^* \frac{\partial \mathcal{L}}{\partial {h_T}^*} \tag{131}$$

Denoting the real and imaginary parts of h_t as h_{tr} and h_{ti}, respectively, we can express $\frac{\partial h_T}{\partial {h_t}^*}$ and $\frac{\partial h_T}{\partial h_t}$ using Eqs. 19-20 as

$$\frac{\partial h_T}{\partial h_t} = \frac{1}{2} \left(\frac{\partial h_T}{\partial h_{tr}} - i \frac{\partial h_T}{\partial h_{ti}} \right), \tag{132}$$

$$\frac{\partial h_T}{\partial {h_t}^*} = \frac{1}{2} \left(\frac{\partial h_T}{\partial h_{tr}} + i \frac{\partial h_T}{\partial h_{ti}} \right). \tag{133}$$

Note that by the design of our network based on the unitary transformation at each layers, we can express h_T as

$$h_T = (W_T W_{T-1} \cdots W_t) h_t. \tag{134}$$

Denoting $(W_T W_{T-1} \cdots W_t)$ as U_t which is a unitary matrix (because the product of unitary matrices is unitary), and its real part and imaginary part as U_{tr} and U_{ti}, respectively, Eq. (134) translates to

$$h_T = U h_t = (U_{tr} h_{tr} - U_{ti} h_{ti}) + i \left(U_{ti} h_{tr} - U_{tr} h_{ti} \right). \tag{135}$$

A straightforward calculation of Eq. (132)-(133) using Eq. (21) leads to

$$\frac{\partial h_T}{\partial h_t} = U_t, \quad \frac{\partial h_T}{\partial h_t^*} = 0. \tag{136}$$

Thus our main chain rule (Eq. (131)) simplifies to

$$\frac{\partial \mathcal{L}}{\partial {h_t}^*} = U_t^* \frac{\partial \mathcal{L}}{\partial {h_T}^*} \tag{137}$$

Since U_t is unitary, U_t^* is also unitary. Taking norms on both sides yields

$$\left\| \frac{\partial \mathcal{L}}{\partial {h_t}^*} \right\|_2 = \left\| \frac{\partial \mathcal{L}}{\partial {h_T}^*} \right\|_2 \qquad \forall t, T \gg t, \tag{138}$$

which proves Eq. (129). Following a similar approach, one can easily prove Eq. (130). □

C.2 CONSTRUCTION OF INTRINSIC TWO-DIMENSIONAL SLICES OF THE STIEFEL MANIFOLD

For an n-qubit circuit dataset, to gain geometric insight into the optimization dynamics beyond scalar metrics, we analyze intrinsic two-dimensional slices of the Stiefel manifold $\mathcal{V}_N(\mathbb{C}^N)$, where $N = 2^n$. Since $\mathcal{V}_N(\mathbb{C}^N)$ coincides with the unitary group $U(N)$ and forms a compact Lie group, its Lie algebra consists of skew-Hermitian matrices,

$$\mathfrak{u}(N) = \{ \Omega \in \mathbb{C}^{N \times N} : \Omega^\dagger = -\Omega \}. \tag{139}$$

Near a target unitary W_{opt}, each iterate can be approximated as

$$W_k \approx \exp(\Omega_k) \, W_{\text{opt}}, \tag{140}$$

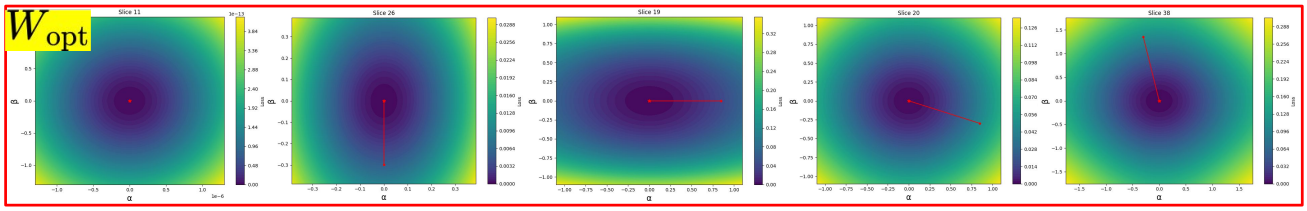


Figure 9: Two-dimensional slices of the Stiefel manifold for 2-qubit Bell matrix dataset for no noise case, and plotted across different skew-Hermitian pairs (A, B). All slices are centered at target unitary W_{opt} due to the noiseless case (as both W_{opt} and W_{final} slices are similar). It can be seen that not all pairs of basis pairs projected at W_{opt} need not give a trajectory in that slice (see Panel 1). However, if the optimization happens, we may find several pairs where trajectory moves dominantly along one basis (Panels 2 and 3), and others where trajectories move along both bases (Panels 4 and 5).

where Ω_k is skew-Hermitian element governing the local displacement which is obtained (whenever defined) via:

$$\Omega_k \approx \log\big(W_k W_{\text{opt}}^\dagger\big), \tag{141}$$

which provides exponential coordinates of W_k relative to W_{opt}. The approximation reflects the local exponential coordinate structure of the unitary group. To construct a two-dimensional intrinsic slice, we permute two skew-Hermitian generators $A, B \in \mathfrak{u}(N)$ w. The Lie algebra element Ω_k is then projected onto the real two-dimensional subspace spanned by $\{A, B\}$ using the real Frobenius inner product:

$$\langle X, Y\rangle = \text{Re}\,\big(\,\text{tr}(X^\dagger Y)\big), \tag{142}$$

The intrinsic coordinates are defined as:

$$\alpha = \text{Re}(\langle \Omega_k, A\rangle), \qquad \beta = \text{Re}(\langle \Omega_k, B\rangle), \tag{143}$$

These real scalars represent the components of Ω_k along the chosen generators and provide local coordinates for visualization. Retaining only this projected component yields the two-dimensional representation:

$$W_k(\alpha, \beta) \approx \exp(\alpha A + \beta B)\, W_{\text{opt}}, \tag{144}$$

Although the full Ω_k generally contains additional Lie algebra components, restricting to $\alpha A + \beta B$ provides a faithful low-dimensional visualization of the local geometry. Because $\alpha A + \beta B$ is skew-Hermitian for all real (α, β), the exponential preserves unitarity exactly, ensuring that the slice remains entirely within the manifold.

The corresponding restricted loss surface is:

$$\mathcal{L}_{A,B}(\alpha, \beta) = \frac{1}{M} \sum_{i=1}^{M} \|W_k(\alpha, \beta) x_i - y_i\|_2^2, \tag{145}$$

which is evaluated over a grid in (α, β) to generate the heatmap of the manifold slice while holding all remaining directions fixed. The plotted trajectory (α, β) represents the projection of the true optimization path onto the chosen two-dimensional Lie subspace. In the ideal case without noise, convergence implies $\Omega_k \to 0$, and hence $(\alpha, \beta) \to (0, 0)$, corresponding exactly to the optimal unitary W_{opt}. If the final unitary W_{final} were not optimal, the path would terminate at a point $(\alpha, \beta) \neq (0, 0)$. A similar approach is followed to find the slices at W_{final} (with W_{opt} in the above equations changed to W_{final}). It is to be noted that not all permutations of the skew-Hermitian basis pairs can capture the trajectory of optimization (see Fig. 9). In our experiments, we considered slices where α and β are significant to gain better insights of the convergence utilizing a full 2D plane.

D COMPLEXITY ANALYSIS

We analyze the computational and memory requirements of Algorithm 1 for training an n-qubit quantum circuit represented as a deep unitary network. The network consists of L hidden layers of $N \times N$ unitary weight matrices, where $N = 2^n$, and is trained on M input-output quantum state pairs per iteration.

D.0.1 Time Complexity Derivation

Let us consider the operations performed during one iteration (step k) of Algorithm 1:

1. **Forward pass:** For each input state $|\psi\rangle$, we compute:

$$|\psi'\rangle = W_{L+1}^{(k)} W_L^{(k)} \cdots W_1^{(k)} |\psi\rangle . \tag{146}$$

   For M input states and L layers, this requires $(L+1)M$ matrix-vector multiplications. Each matrix-vector multiplication with an $N \times N$ matrix takes $\mathcal{O}(N^2)$ operations. Thus, the total cost is: $\mathcal{O}(LMN^2)$.

2. **Loss computation:** Computing the squared norm difference between two N-dimensional vectors requires $\mathcal{O}(N)$ operations. For M examples: $\mathcal{O}(MN)$.
   This term is dominated by the forward pass and will be absorbed in subsequent analysis.

3. **Gradient computation:** Using backpropagation through the unitary layers, for each of the M examples and L layers, we need: (a) $\mathcal{O}(N^2)$ for the outer product $\delta_i^{(j)}$; (b) $\mathcal{O}(N^2)$ for backpropagating the error through the adjoint matrix The total gradient computation cost is therefore: $\mathcal{O}(LMN^2)$.

4. **Skew-Hermitian matrix computation :** For each layer i, we compute:

$$A_i = (\nabla_{W_i}\mathcal{L})W_i^\dagger - W_i(\nabla_{W_i}\mathcal{L})^\dagger. \tag{147}$$

   This involves two matrix-matrix multiplications of $N \times N$ matrices, each costing $\mathcal{O}(N^3)$. For L layers: $\mathcal{O}(LN^3)$.

5. **Cayley transform update:** The update:

$$W_i^{(k+1)} = \left(I + \frac{\lambda}{2}A_i\right)^{-1} \left(I - \frac{\lambda}{2}A_i\right) W_i^{(k)}, \tag{148}$$

   requires: (a) Matrix inversion: $\mathcal{O}(N^3)$; (b) Two matrix-matrix multiplications: $\mathcal{O}(N^3)$ each.
   For L layers, this gives: $\mathcal{O}(LN^3)$.

The dominant terms are $\mathcal{O}(LMN^2)$ from the forward pass and gradient computation, and $\mathcal{O}(LN^3)$ from the matrix operations in the update step. Thus, the time complexity per iteration is:

$$\boxed{\mathcal{O}\left(LMN^2 + LN^3\right).} \tag{149}$$

D.0.2 Space Complexity Derivation

The memory requirements are determined by:

1. **Weight matrices storage:** L unitary matrices of size $N \times N$, requiring: $\mathcal{O}(LN^2)$.
2. **Intermediate state storage:** For efficient backpropagation, we need to store the input to each layer for each training example. For M examples and L layers, with each state being an N-dimensional vector: $\mathcal{O}(MLN)$.
3. **Gradient storage:** Gradients for L weight matrices, each $N \times N$: $\mathcal{O}(LN^2)$.
4. **Temporary matrices:** During the update step, we need to store A_i, the inverted matrix, and intermediate products, each $N \times N$. Since these can be reused layer by layer: $\mathcal{O}(N^2)$.

The dominant terms are the weight storage and intermediate state storage. Thus, the space complexity per iteration is:

$$\boxed{\mathcal{O}\left(LN^2 + MLN\right).} \tag{150}$$

This analysis provides the theoretical foundation for understanding the scalability of Stiefel manifold optimization in quantum circuit training applications.

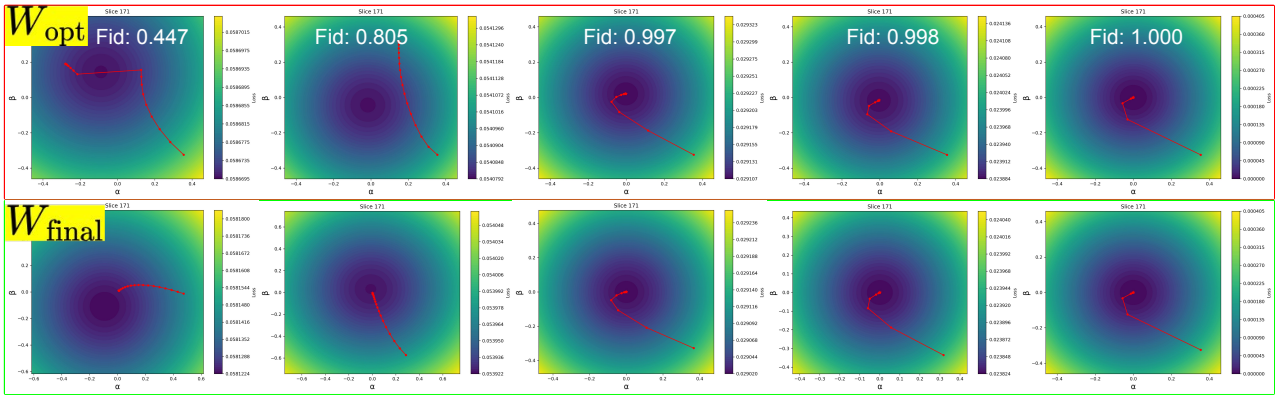
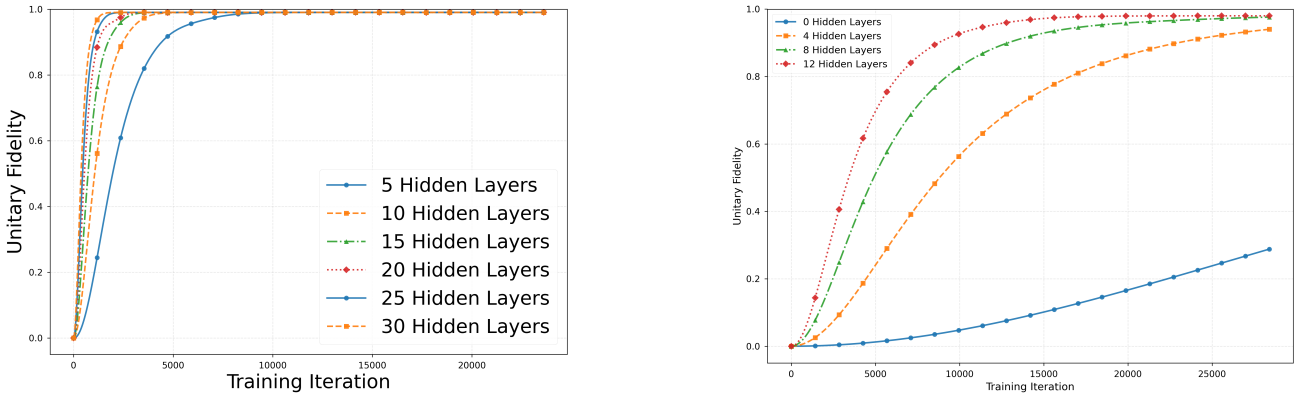


Figure 10: Two-dimensional slices of the Stiefel manifold showing optimization trajectories under different noise levels (high (left) to low(right)). Top row: slices centered at target unitary W_{opt}. Bottom row: same trajectories re-centered at final learned unitary W_{final}. Under high noise, the W_{opt} slice shows convergence to $(\alpha, \beta) \neq (0, 0)$, indicating a different minimizer, while the W_{final} slice confirms smooth local convergence. As noise decreases, both slices converge to the origin, demonstrating perfect recovery in the noiseless case.



(a) 7-qubit random circuit

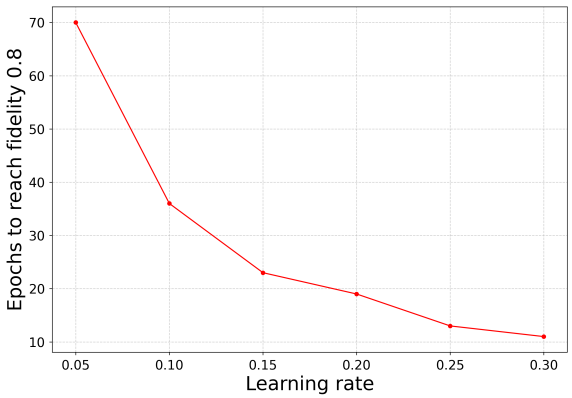
(b) 8-qubit random circuit

Figure 11: Fidelity vs training iterations under phase-damping noise for varying network depths for 7-qubit and 8-qubit random quantum circuit with $\lambda = 0.01$. Deeper architectures achieve faster convergence and higher final fidelity.

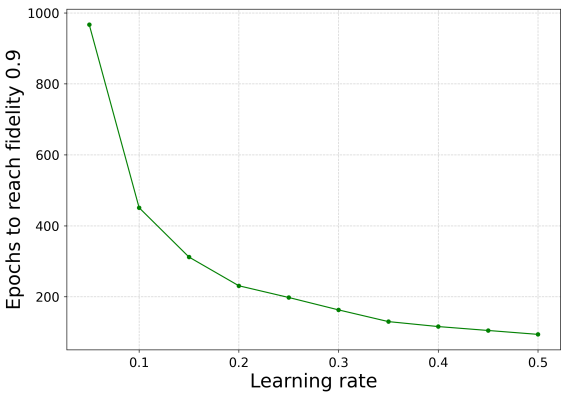
E ADDITIONAL EXPERIMENTS

E.1 VISUALIZATION

Figure 2 and Figure 10 depict two-dimensional intrinsic slices of the *same* high-dimensional optimization landscape on the Stiefel manifold $\mathcal{V}_N(\mathbb{C}^N)$, constructed from the identical loss function and optimization trajectory. The difference arises solely from the choice of skew-Hermitian generators $\{A, B\}$ used to define the Lie algebra projection subspace. Since different pairs of generators span different real two-dimensional planes inside $\mathfrak{u}(N)$, each slice represents a different geometric cross-section of the same terrain. Consequently, curvature and apparent trajectory bending may differ visually, even though the underlying optimization dynamics are identical. More specifically, the two-dimensional slices (from high noise on the left to low noise on the right) illustrate how the optimization trajectory behaves when visualized in coordinates centered either at W_{opt} (top row) or at W_{final} (bottom row). Under high noise, the slice centered at W_{opt} shows the trajectory converging to a point $(\alpha, \beta) \neq (0, 0)$, indicating that the learned unitary differs from the ground-truth minimizer. In contrast, when the same trajectory is re-centered at W_{final}, the slice exhibits smooth local convergence to the origin, confirming that W_{final} is indeed a local minimizer of the noisy objective. As the noise level decreases, both visualizations progressively align, and in the noiseless case the trajectories in both coordinate systems converge to $(0, 0)$, demonstrating exact recovery of W_{opt}.



(a) 5-qubit random circuit



(b) 7-qubit random circuit

Figure 12: Number of epochs required to reach a certain fidelity vs. learning rate under depolarizing noise ($p = 0.01$).

E.2 EFFECT OF DEPTH ON CONVERGENCE AND DATA REQUIREMENT

To further examine the effect of network depth on convergence, we analyze larger 7- and 8-qubit random circuits under phase-damping noise ($\lambda = 0.01$). Figure 11 shows that for the 7-qubit case (Fig. 11a), increasing the depth from 5 to 30 layers leads to consistently faster convergence and higher final fidelity. A similar trend appears for the 8-qubit circuit (Fig. 12b), where deeper models (4–12 layers) outperform shallow ones, which converge slowly and plateau at lower fidelity. These results confirm that the geometric optimization framework effectively exploits increased depth to improve convergence and fidelity, with gains persisting even at larger system sizes.

We analyze the interplay between network depth and dataset size by plotting final fidelity versus k, where the dataset contains $k \times 2^n$ samples. Figure 13 shows results for a 4-qubit random circuit (Figure 13a) and a 6-qubit QFT circuit (Figure 13b) under depolarizing noise ($p = 0.01$), using networks with 0, 5, and 10 hidden layers. In both cases, deeper networks require substantially fewer samples to achieve high fidelity. For example, in the 4-qubit circuit, the 0-layer model needs $k \approx 10$ to reach near-unity fidelity, whereas the 10-layer model achieves similar performance with $k \approx 5$. These results highlight that increased depth improves both convergence and sample efficiency, which is crucial in data-limited quantum settings.

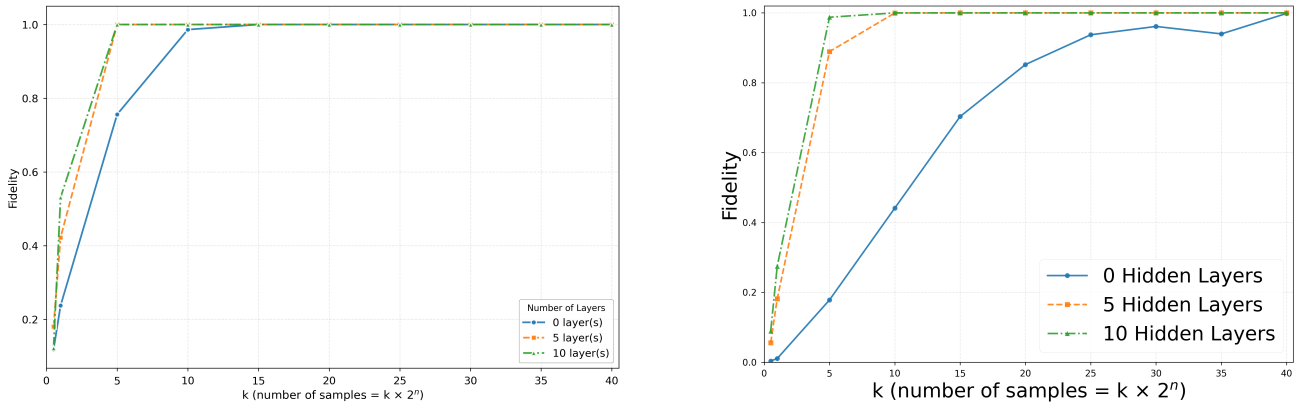
E.3 EFFECT OF LEARNING RATE AND INITIALIZATION

We examine the effect of learning rate on convergence by measuring the epochs required to reach a target fidelity under depolarizing noise ($p = 0.01$). Figure 12 shows results for 5- and 7-qubit random circuits (Figs. 12a and 12b). In both cases, higher learning rates consistently reduce the number of epochs needed to reach the target fidelity, with no observed instability. This monotonic trend indicates a well-behaved optimization landscape on the Stiefel manifold, where larger step sizes accelerate convergence without compromising stability, demonstrating the robustness of our geometric framework.

We investigate the sensitivity of our Stiefel manifold optimization to the initial guess of the unitary matrix. To test this, we perform multiple training runs on a 5-qubit random circuit task under two different noise conditions, each starting from a different randomly initialized unitary matrix. The target unitary is fixed, and all hyperparameters are kept identical across runs within each noise setting.

Figures 14 and 15 present the results for low-noise and high-noise scenarios, respectively. The left panels (Figs. 14a and 15a) show the initial fidelity between the randomly initialized weight matrix and the true target unitary across five different seeds. In all cases, the initial fidelity is extremely low (on the order of 10^{-3} or less), confirming that the starting point is far from the target in the unitary space. The right panels (Figs. 14b and 15b) display the fidelity during training over 80,000 epochs (epochs are shown in units of 10^3).

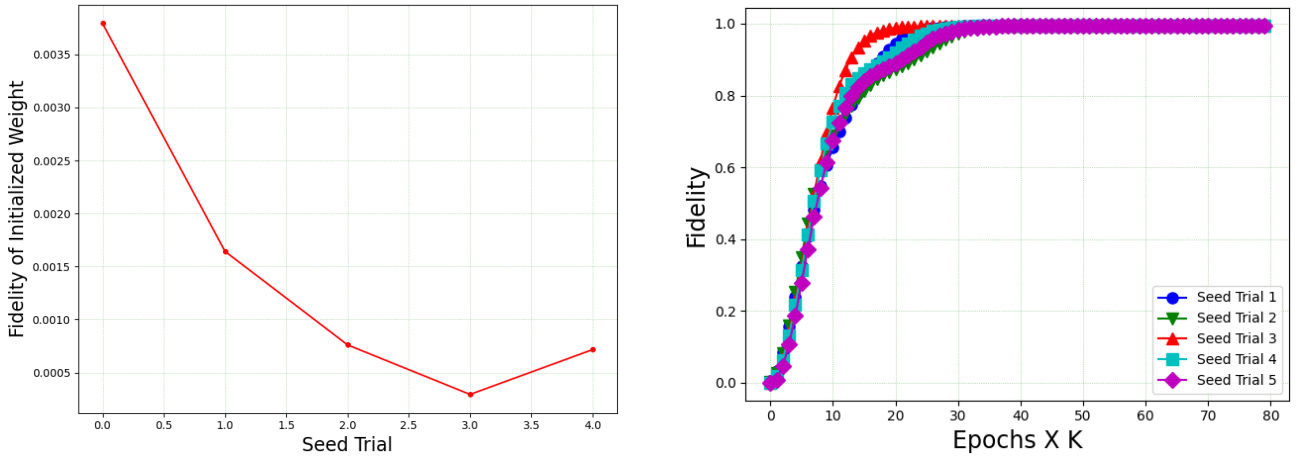
Despite the wide disparity in initial fidelities and the complete randomness of the starting matrices, all runs converge smoothly to near-perfect fidelity (> 0.99) regardless of the noise level. The learning curves are nearly indistinguishable after the several epochs, demonstrating that the optimization is essentially insensitive to initialization. This behavior holds for both low-noise and high-noise regimes, indicating that the Stiefel manifold formulation, combined with Cayley updates, provides a robust gradient flow that consistently guides the parameters toward the global minimum, even when starting far



(a) 4-qubit random circuit

(b) 6-qubit QFT circuit

Figure 13: Fidelity vs. k (dataset size $= k \times 2^N$) under depolarizing noise ($p = 0.01$) for 4-qubit random and 6-qubit QFT circuit with networks containing 0, 5, and 10 hidden layers.



(a) Fidelity of initialization (low noise)

(b) Fidelity during training (low noise)

Figure 14: Effect of initialization on a 5-qubit random circuit under low noise. Initial fidelities are negligible, yet all seeds converge to the target.

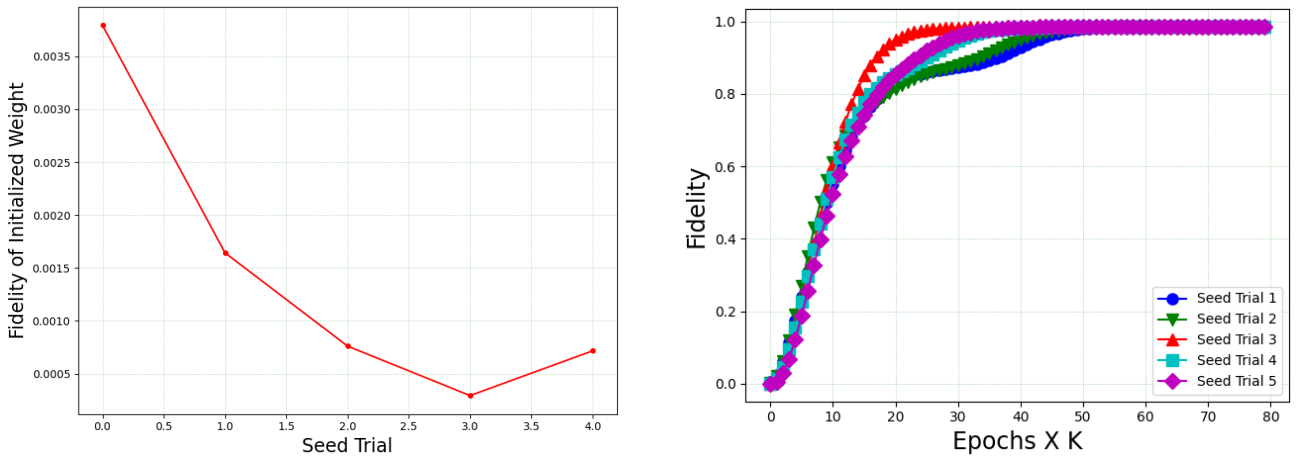
from the solution and under significant noise.

E.4 SCALABILITY EXPERIMENTS

We further validate the scalability of our Neumann series approximation on a 7-qubit random circuit, as shown in Fig. 16. The training time (Fig. 16a) exhibits a modest increase as M grows from low orders, but rises substantially when computing the exact inverse ($M = \infty$). Critically, the fidelity (Fig. 16b) remains consistently high across all truncation orders, including the exact case. The unitary error (Fig. 16c) is somewhat elevated at the lowest truncation order, but rapidly diminishes to negligible levels for $M \geq 2$. These findings reinforce our conclusion that low-order approximations ($M = 2$ or 3) provide an excellent trade-off between computational efficiency and accuracy, confirming the scalability of our geometric optimization framework to larger quantum systems.

E.5 IMPACT OF NOISY TRAINING PAIRS AND CONVERGENCE IN QUANTUM CIRCUIT SPACE

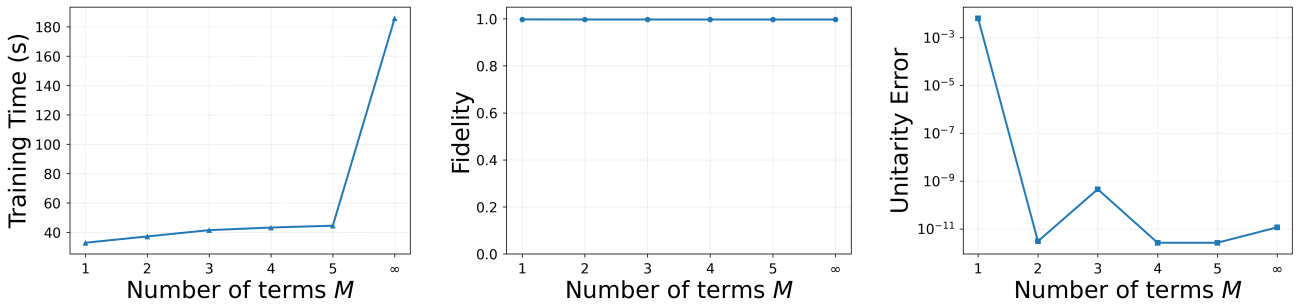
In all previous experiments, the entire training set was corrupted by noise in a probabilistic fashion. Here we investigate a more realistic scenario where only a subset of the training data is corrupted. Instead of applying a standard noise model,



(a) Fidelity of initialization (high noise)

(b) Fidelity during training (high noise)

Figure 15: Effect of initialization on a 5-qubit random circuit under high noise. The same robust convergence is observed, demonstrating noise resilience.

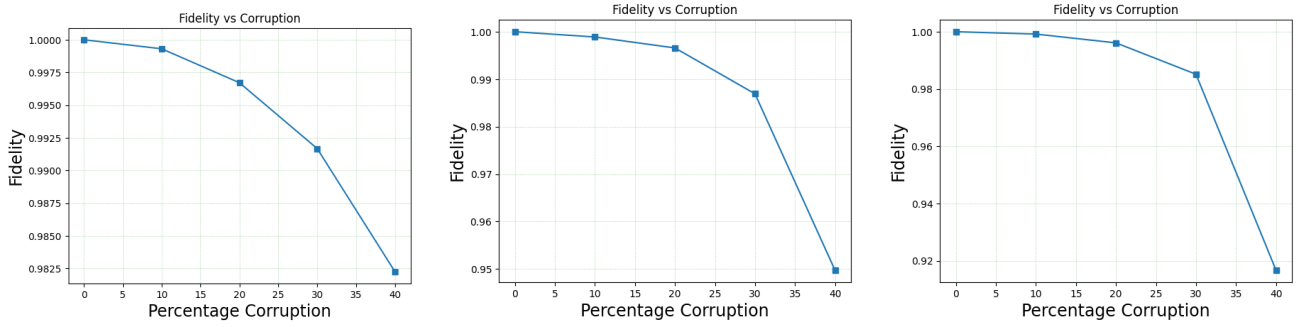


(a) Training time

(b) Fidelity

(c) Unitary error

Figure 16: Scalability study showing (a) training time, (b) fidelity, and (c) unitary error as functions of the number of Neumann-series terms M for a 7-qubit random circuit dataset



(a) 2-qubit

(b) 3-qubit

(c) 4-qubit

Figure 17: Fidelity as a function of the percentage of corrupted training pairs for 2-, 3-, and 4-qubit random circuits. The graceful decline shows robustness to increasing amounts of noisy data.

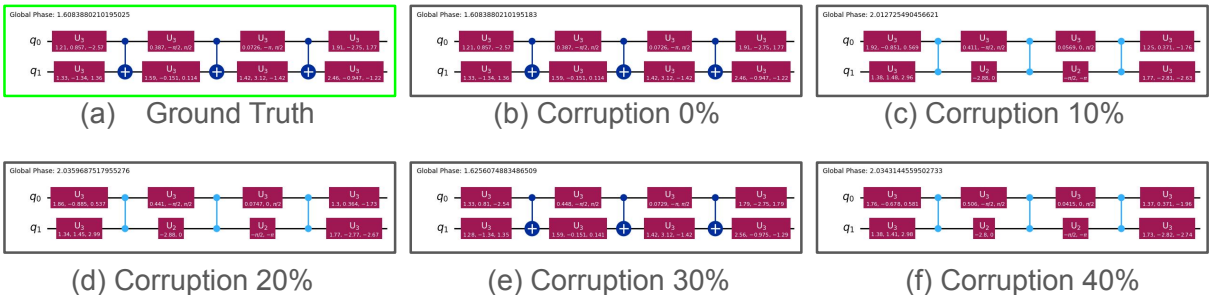


Figure 18: Transpiled quantum circuits for the 2-qubit case (corresponding to Fig. 17a) showing the ground truth and learned unitaries at increasing corruption levels. The structural similarity across all levels indicates stable convergence to a consistent region of the unitary manifold.

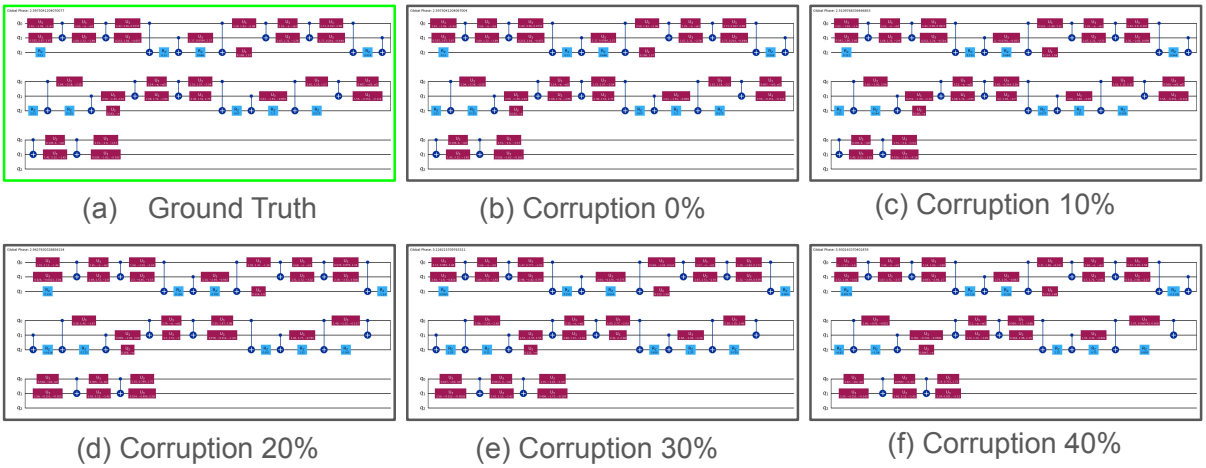


Figure 19: Transpiled quantum circuits for the 3-qubit case (corresponding to Fig. 17b). Despite varying corruption, the circuits retain a recognizable structural pattern, confirming reliable convergence.

we deliberately corrupt only selected data points by randomly permuting the entries of their output vectors. This operation destroys the relational structure between input and output while preserving the norm, simulating a severe loss of information. For a fixed total dataset size S (we use $S = 300$ for 2-qubit, $S = 500$ for 3-qubit, and $S = 1000$ for 4-qubit random circuits), we introduce corruption into a varying number m of the training pairs, leaving the remaining $S - m$ samples uncorrupted. We then train our Stiefel manifold model and record the final fidelity as a function of the corruption percentage $m/S \times 100\%$, keeping all hyperparameters identical to the previous settings.

Figure 17 shows the fidelity achieved for each system size as the corruption percentage increases. In all cases, the fidelity decreases gracefully as more corrupted samples are introduced. The decline is smooth rather than abrupt, indicating that the geometric optimization remains stable even when a significant fraction of the data is inconsistent. For example, with up to 20% corruption the fidelity remains above 0.99 for all three qubit counts, and only drops substantially beyond 30%. This graceful degradation demonstrates the robustness of the Stiefel manifold approach to imperfect training data.

To gain further insight into the convergence behavior, we transpile the learned unitary matrices into quantum circuits using Qiskit's standard transpilation pipeline. Figures 18-20 display the resulting circuits for the 2-, 3-, and 4-qubit cases, respectively, for corruption levels 0%, 10%, 20%, 30%, and 40% (of Fig. 17). The ground truth circuit is also shown for reference. A striking observation is the structural similarity of the transpiled circuits across all corruption levels for a given qubit count. Despite the decrease in numerical fidelity at higher corruption rates, the circuit topology—the arrangement of gates and the pattern of entangling operations—remains largely unchanged. This indicates that the optimization consistently converges to the same region of the unitary manifold, and the degradation in fidelity is due to small parameter deviations rather than a collapse to a completely different minimum. Even at 40% corruption, where the fidelity is notably lower, the circuit retains the essential structure of the ground truth, confirming that the learning process remains stable and reliable.

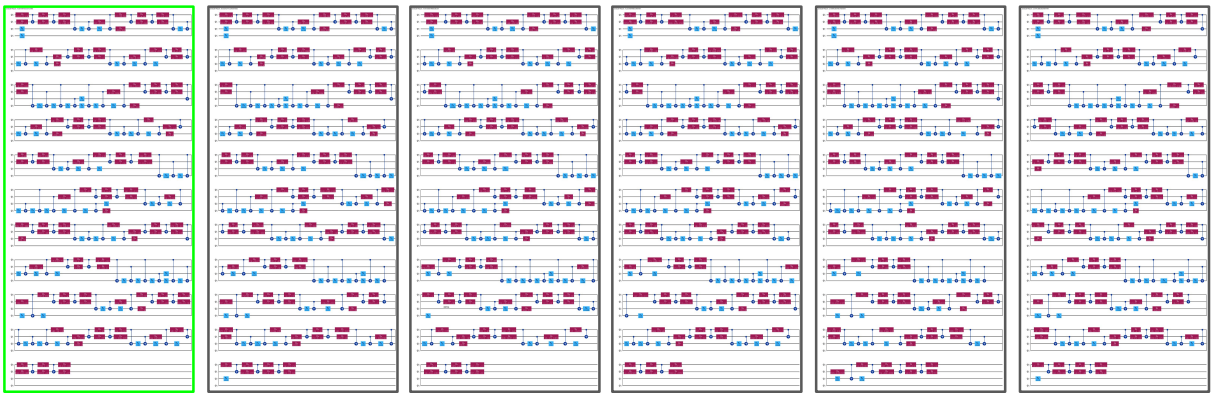


Figure 20: Transpiled quantum circuits for the 4-qubit case (corresponding to Fig. 17c). The consistent circuit topology across corruption levels demonstrates that the optimization reliably converges to the target region of the landscape.

F IMPLEMENTATION DETAILS

F.1 QUANTUM NOISE MODELS

In this section, we introduce the fundamental concepts of quantum states and channels, explain the simulation methodology used to generate noisy data, and provide a detailed mathematical description of the three noise models employed in our study.

F.1.1 Quantum States and Channels

A quantum system is described by a state vector in a complex Hilbert space. A *pure state* is represented by a unit vector $|\psi\rangle$. Its corresponding more generalized representation, density matrix is $\rho = |\psi\rangle\langle\psi|$, which contains all information about the state. More generally, a statistical mixture of pure states $\{|\psi_i\rangle\}$ with probabilities p_i is described by a *density matrix* $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$. Density matrices can represent both pure and mixed states; a state is pure if and only if $\mathrm{Tr}(\rho^2) = 1$ (Nielsen and Chuang [2002]).

The evolution of a quantum system under noise is described by a *quantum channel*, a completely positive trace-preserving (CPTP) linear map $\mathcal{E}$. Any such channel can be represented using the *Kraus operator-sum representation* :

$$\mathcal{E}(\rho) = \sum_k E_k \rho E_k^\dagger, \tag{151}$$

where the Kraus operators $\{E_k\}$ satisfy $\sum_k E_k^\dagger E_k = I$ (Nielsen and Chuang [2002]). This representation is fundamental for describing both coherent and incoherent processes.

In this work, we focus on pure-state simulations. While a noisy channel acting on a pure state generally produces a mixed state, we circumvent the need for density matrices by employing the Monte Carlo trajectory method, which stochastically unravels the channel into an ensemble of pure-state trajectories. As a result, our dataset consists exclusively of pure states-each corresponding to a single noisy realization, rather than mixed states.

F.1.2 Monte Carlo Trajectory Method

The statevector simulator in Qiskit Aer (Aleksandrowicz et al. [2022]) implements the Monte Carlo trajectory method (also known as the stochastic wavefunction method) to simulate noisy quantum circuits. The core idea, illustrated in Fig. 21, is to replace the deterministic evolution of a density matrix with a stochastic evolution of a pure state, where at each noise location one Kraus operator is applied randomly according to a probability distribution derived from the current state.

For a given set of Kraus operators $\{E_i\}$ acting on a pure state $|\psi\rangle$, the algorithm proceeds as follows:

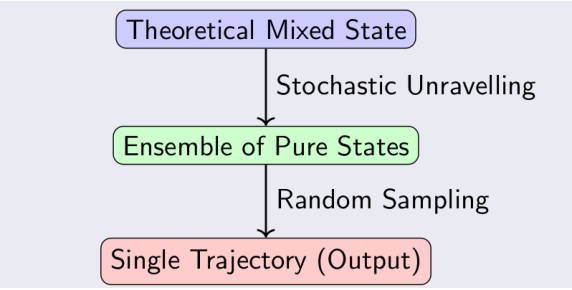


Figure 21: Conceptual illustration of the Monte Carlo trajectory method. A mixed state ρ (theoretical output of a noise channel) is unravelled into an ensemble of pure states $\{|\psi_i\rangle\}$. A single trajectory randomly selects one pure state from this ensemble.

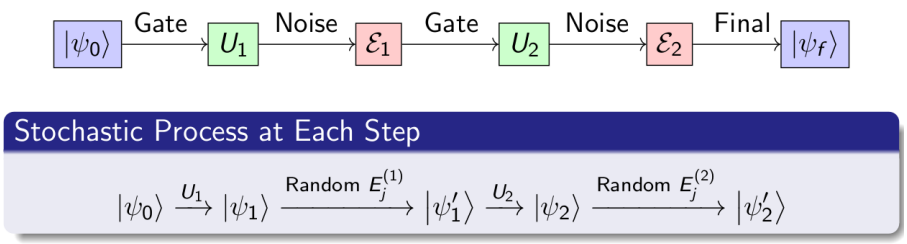


Figure 22: Circuit-level view of multiple noise operations. At each step, an independent random Kraus operator is chosen, leading to a specific trajectory through the space of possible outcomes. The final state $|\psi_f\rangle$ is a pure state representing one noisy realization.

1. Compute the probabilities $p_i = \langle\psi|E_i^\dagger E_i|\psi\rangle$. By the trace-preserving condition, $\sum_i p_i = 1$.
2. Generate a uniform random number $r \in [0, 1]$.
3. Select the Kraus operator E_j such that $\sum_{k=0}^{j-1} p_k \leq r < \sum_{k=0}^{j} p_k$.
4. Apply the selected operator: $|\psi'\rangle_{\text{temp}} = E_j|\psi\rangle$.
5. Renormalize to obtain the new pure state: $|\psi'\rangle = |\psi'\rangle_{\text{temp}}/\||\psi'\rangle_{\text{temp}}\|$.

This process is repeated for every noise operation in the quantum circuit. Each complete simulation run follows a unique sequence of random choices, producing one pure-state trajectory (Fig. 22). Averaging over many such trajectories would recover the density matrix evolution $\rho' = \sum_i E_i|\psi\rangle\langle\psi|E_i^\dagger$, but each individual run yields a pure state. This is precisely how our dataset is generated: for each input pure state $|\psi_i^{\text{clean}}\rangle$, we simulate the circuit once with the specified noise model and record the resulting pure state $|\psi_i^{\text{noisy}}\rangle$. Consequently, our dataset consists of pure-state samples from the noisy distribution, not mixed states.

F.1.3 Noise Channel Definitions

We now describe the three noise channels used in our simulations. Each channel is defined by its Kraus operators and a noise-strength parameter. The parameters λ (phase damping), γ (amplitude damping), and p (depolarizing) are varied to generate datasets with different noise levels.

Phase Damping Channel (parameter λ). Phase damping, also known as dephasing, models the loss of quantum coherence without energy relaxation. In the density matrix representation,

$$\rho = \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix}, \tag{152}$$

the action of the phase damping channel is to preserve the diagonal elements while decaying the off-diagonal elements by a factor $1-\lambda$:

$$\mathcal{E}_{\text{PD}}(\rho) = \begin{pmatrix} \rho_{00} & (1-\lambda)\rho_{01} \\ (1-\lambda)\rho_{10} & \rho_{11} \end{pmatrix}, \tag{153}$$

with $\lambda \in [0,1]$. When $\lambda = 0$ the channel is the identity; when $\lambda = 1$ the off-diagonals are completely suppressed, resulting in a fully dephased (classical) state.

Amplitude Damping Channel (parameter γ). Amplitude damping describes energy dissipation, such as spontaneous emission from the excited state $|1\rangle$ to the ground state $|0\rangle$. The transformation of the density matrix elements under this channel is given by

$$\mathcal{E}_{\text{AD}}(\rho) = \begin{pmatrix} \rho_{00} + \gamma\rho_{11} & \sqrt{1-\gamma}\,\rho_{01} \\ \sqrt{1-\gamma}\,\rho_{10} & (1-\gamma)\rho_{11} \end{pmatrix}, \tag{154}$$

where $\gamma \in [0,1]$ is the probability of energy loss. For $\gamma = 0$ the channel is identity; for $\gamma = 1$ the qubit deterministically decays to $|0\rangle$, yielding $\mathcal{E}_{\text{AD}}(\rho) = |0\rangle\langle 0|$.

Depolarizing Channel (parameter p). Depolarizing noise replaces the qubit state with the maximally mixed state $I/2$ with probability p, and leaves it unchanged with probability $1-p$. Its action on a density matrix is

$$\mathcal{E}_{\text{depol}}(\rho) = (1-p)\,\rho + p\,\frac{I}{2}, \tag{155}$$

with $p \in [0,1]$. Equivalently, in the Bloch sphere representation, the Bloch vector $\mathbf{r}$ (such that $\rho = (I + \mathbf{r}\cdot\boldsymbol{\sigma})/2$) is scaled by a factor $1-p$:

$$\mathbf{r} \longrightarrow (1-p)\,\mathbf{r}. \tag{156}$$

Thus, as p increases, the state becomes more mixed, converging to the completely mixed state at $p = 1$.

F.1.4 Dataset Generation and Implications

Using the Qiskit Aer simulator with the statevector method and the Monte Carlo trajectory approach, we generate a dataset consisting of tuples

$$\mathcal{D} = \left\{ \left(|\psi_i\rangle, |\psi_i^{\text{clean}}\rangle, |\psi_i^{\text{noisy}}\rangle \right) \right\}_{i=1}^{M}, \tag{157}$$

where $|\psi_i\rangle$ denotes an identifying label or input encoding, $|\psi_i^{\text{clean}}\rangle$ is the ideal (noise-free) output state, and $|\psi_i^{\text{noisy}}\rangle$ is the output of a single noisy trajectory. Because each trajectory yields a pure state, our model is trained on pure-state samples drawn from the probability distribution induced by the noise, not on the mixed-state averages.

This approach is essential for developing machine learning models that can predict the stochastic outcomes of noisy quantum circuits. By training on individual trajectories, the model learns to capture the full probabilistic nature of the noise, which is crucial for tasks such as error mitigation and quantum control.

F.2 IMPLEMENTATION OF ORTHOGONALIZATION METHODS

This appendix provides detailed mathematical descriptions of the five orthogonalization methods considered in our comparative study for enforcing unitarity constraints in neural networks.

F.2.1 Gram-Schmidt (GS) Orthogonalization

Given a non-unitary weight matrix $W \in \mathbb{C}^{N\times N}$ with column vectors $\{\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_N\}$, the Gram-Schmidt process constructs an orthonormal set of vectors $\{\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_N\}$ that spans the same column space. The algorithm proceeds iteratively:

$$\mathbf{v}_1 = \mathbf{w}_1, \tag{158}$$

$$\mathbf{u}_1 = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|}, \tag{159}$$

$$\mathbf{v}_k = \mathbf{w}_k - \sum_{j=1}^{k-1} \text{proj}_{\mathbf{u}_j}(\mathbf{w}_k), \quad k = 2, 3, \ldots, N \tag{160}$$

$$\mathbf{u}_k = \frac{\mathbf{v}_k}{\|\mathbf{v}_k\|}, \tag{161}$$

where the projection operator is defined as $\text{proj}_{\mathbf{u}}(\mathbf{w}) = \langle \mathbf{w}, \mathbf{u} \rangle \mathbf{u}$ with the complex inner product $\langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^\dagger \mathbf{b}$. The resulting unitary matrix is $U = [\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_N]$.

The computational complexity of Gram-Schmidt is $O(N^3)$ for an $N \times N$ matrix. While straightforward to implement, classical Gram-Schmidt suffers from numerical instability when the input matrix is ill-conditioned, as rounding errors can accumulate and lead to loss of orthogonality.

F.2.2 Modified Gram-Schmidt (MGS) Orthogonalization

Modified Gram-Schmidt improves the numerical stability of the classical algorithm by orthogonalizing against all previous vectors at each step. Given $W = [\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_N]$, the algorithm proceeds as follows:

1. Initialize: $\mathbf{v}_1^{(1)} = \mathbf{w}_1$
2. For $k = 1, 2, \ldots, N$:

$$r_{kk} = \|\mathbf{v}_k^{(k)}\|, \tag{162}$$

$$\mathbf{q}_k = \frac{\mathbf{v}_k^{(k)}}{r_{kk}}, \tag{163}$$

$$\text{For } j = k+1, \ldots, N: \quad r_{kj} = \langle \mathbf{q}_k, \mathbf{v}_j^{(k)} \rangle, \tag{164}$$

$$\mathbf{v}_j^{(k+1)} = \mathbf{v}_j^{(k)} - r_{kj} \mathbf{q}_k. \tag{165}$$

The key difference from classical Gram-Schmidt is that MGS updates the remaining vectors immediately after computing each orthonormal vector, which reduces error accumulation.

F.2.3 Singular Value Decomposition (SVD) based Orthogonalization

Given a matrix $W \in \mathbb{C}^{N \times N}$, its singular value decomposition is:

$$W = U \Sigma V^\dagger, \tag{166}$$

where U and V are unitary matrices, and Σ is a diagonal matrix of singular values.

To obtain the closest unitary matrix to W in the Frobenius norm, we compute:

$$U_{\text{unitary}} = U V^\dagger. \tag{167}$$

This matrix minimizes $\|W - X\|_F$ subject to $X^\dagger X = I$. The algorithm proceeds as:

1. Compute the SVD: $W = U \Sigma V^\dagger$.
2. Extract the unitary factor: $U_{\text{unitary}} = U V^\dagger$.

The SVD approach provides optimal approximation properties: among all unitary matrices, $UV^\dagger$ minimizes both the Frobenius norm $\|W - X\|_F$ and the spectral norm $\|W - X\|_2$. This makes it particularly suitable for applications where approximation quality is critical.

F.2.4 Lie Algebra based Orthogonalization

An alternative approach to enforce unitarity is to parameterize the unitary matrix via its Lie algebra. Given a matrix $W \in \mathbb{C}^{N\times N}$, we first extract its lower triangular part $L = \mathrm{tril}(W)$. From L, we construct a skew-Hermitian matrix:

$$B = L - L^\dagger, \tag{168}$$

which satisfies $B^\dagger = -B$. The matrix exponential of a skew-Hermitian matrix yields a unitary matrix:

$$U_{\text{unitary}} = \exp(B). \tag{169}$$

This follows from the property $\exp(B)^\dagger = \exp(B^\dagger) = \exp(-B) = \exp(B)^{-1}$.

The algorithm proceeds as:

1. Extract the lower triangular part: $L = \mathrm{tril}(W)$.
2. Form the skew-Hermitian matrix: $B = L - L^\dagger$.
3. Compute the matrix exponential: $U_{\text{unitary}} = \exp(B)$.

Unlike the SVD method, this approach does not provide the closest unitary matrix in the Frobenius norm. Instead, it maps the lower triangular elements directly to the Lie algebra, ensuring that the resulting matrix is exactly unitary.

F.3 IMPLEMENTATION OF UNITARY PARAMETERIZATIONS

In this work, we implement multiple parameterizations of unitary operators in order to compare their expressive power and trainability for variational quantum process tomography. The target unitary $U \in \mathbb{C}^{N\times N}$ (with $N = 2^n$) is learned from input–output state pairs generated from an unknown quantum process. The following parameterizations are implemented.

F.3.1 Vairational Quantum Parametric Circuit (VQPT)

The parametric quantum circuit is constructed according to the architecture described in Xue et al. [2021]. It consists of interlaced layers of single-qubit rotations and nearest-neighbour CNOT gates. For an n-qubit circuit with depth d, the structure is:

1. An initial single-qubit layer.
2. d repetitions of: (a) A layer of staggered CNOT gates; (b) A single-qubit rotation layer.

Each single-qubit layer contains three rotational gates per qubit:

$$R_z(\theta_1)R_y(\theta_2)R_z(\theta_3), \tag{170}$$

where

$$R_y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}, \tag{171}$$

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}. \tag{172}$$

The total number of trainable parameters is $3n(d+1)$. The unitary corresponding to the circuit is extracted using exact statevector simulation.

F.3.2 Full Givens Parameterization

We implement the dense Givens factorization of a d-dimensional unitary matrix as given in Kiani [2020]. For each pair of indices (i, j) with $i < j$, we construct a 2×2 unitary block

$$q(\theta, \phi) = \begin{pmatrix} \cos\theta & e^{i\phi}\sin\theta \\ -e^{-i\phi}\sin\theta & \cos\theta \end{pmatrix}, \tag{173}$$

which is embedded into the (i, j) subspace of $\mathbb{C}^{d\times d}$.

The total unitary is constructed as a product of such rotations:

$$U = \prod_{i<j} G_{ij}(\theta_{ij}, \phi_{ij}), \tag{174}$$

where each G_{ij} acts non-trivially only on the (i, j) coordinates. This requires $d(d-1)$ real parameters.

F.3.3 Lie Algebra Parameterization

Any unitary $U \in U(d)$ can be written as

$$U = e^{iH}, \tag{175}$$

where H is a Hermitian matrix.

We parameterize H directly: (a) d real diagonal entries; (b) two real parameters for each off-diagonal pair (i, j).

Explicitly,

$$H_{ii} = a_i, \tag{176}$$
$$H_{ij} = a_{ij} + ib_{ij}, \quad i < j, \tag{177}$$
$$H_{ji} = a_{ij} - ib_{ij}. \tag{178}$$

This results in d^2 real parameters. The unitary is obtained via matrix exponential:

$$U = \exp(iH). \tag{179}$$

This approach requires computing a matrix exponential at each optimization step and scales poorly with dimension.

F.4 OUR EXPERIMENTAL CONFIGURATION

All experiments were conducted on a high-performance computing node equipped with NVIDIA H100 NVL GPU with 96 GB of memory. The system used NVIDIA driver version 580.126.09 with CUDA 12.1 support. Experiments were executed on a Linux-based environment. We also successfully ran our 6/7 qubit code on Google Collab. The implementation relies on Python 3.10.19 with the following key libraries:

1. `PyTorch` 2.5.1 (built with CUDA 12.1) for automatic differentiation and GPU-accelerated tensor operations.
2. `Qiskit` 2.2.3 and `Qiskit-Aer` 0.17.2 for quantum circuit generation and statevector simulations.
3. `NumPy` 2.2.6 and `SciPy` 1.15.3 for numerical linear algebra routines.
4. `CuPy` 14.0.0 (optional) for additional GPU-based matrix operations.

CUDA version 12.1 is used throughout. We will release our code upon acceptance.